



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Analysis of the Rock5B VPU Driver

Till Bégue

Bachelor's Thesis – July 6, 2026.
Institute of Information Security,
Secure & Trustworthy Systems Group.

Supervisors: Prof. Dr. Shweta Shinde
M.Sc. Clément Thorens



Abstract

The extraction of Linux kernel drivers requires precise characterization of the kernel-driver boundary: the complete set of kernel functions, data structures, and subsystem interactions on which a driver depends at runtime. This thesis analyzes the Hantro Video Processing Unit (VPU) driver on the Rockchip RK3588 System on Chip (SoC) during AV1 stateless hardware decoding. The evaluation uses a three-stage methodology combining manual call graph tracing, CodeQL static analysis, and **bpftrace** dynamic tracing across the Video4Linux2, Videobuf2, Memory2Memory, and Hantro subsystems.

Manual tracing of `VIDIOC_REQBUFS` establishes a verified ground truth and seeds the static analysis. A CodeQL library resolves both direct calls and indirect calls through operations structs, producing a call graph of 367 reachable functions across 11 ioctl entry points. A complementary structural analysis identifies 34 kernel structures accessed across the subsystem boundary, classified by access direction into callback-injected and query-retrieved structs. Dynamic tracing with **bpftrace** confirms 164 of the 208 probeable functions as active during a 307-frame decode session, identifies 44 statically reachable but never-executed paths, and validates the function-pointer resolution predicates developed during static analysis. The remaining 159 functions could not be probed due to compiler inlining.

The results show that static analysis produces no false negatives against the dynamic trace: every function observed at runtime was predicted by the static analysis. Once bootstrapped against the manual ground truth, the CodeQL library operates independently and can be applied to new entry points or drivers without repeating the manual tracing phase. The automated **bpftrace** script generator further reduces manual effort by deriving dynamic probes directly from static analysis output, allowing the precision of any future analysis to be improved by running the generated scripts against a target workload and eliminating statically predicted but never-executed functions. Together, the three stages yield a precise and reproducible characterization of the kernel-driver boundary, providing the foundation for driver extraction.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

- ☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz¹ verwendet.
- ☒ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

Analysis of the Rock5B VPU Driver

Verfasst von:

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):

Bégué

Vorname(n):

Till

Ich bestätige mit meiner Unterschrift:

- Ich habe mich an die Regeln des «[Zitierleitfadens](#)» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum

Olten 06.07.2026

Unterschrift(en)

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

¹ Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. <https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html> und <https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html> (Änderungen vorbehalten).

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Related Work	2
1.2.1	Hardware Acceleration and the Attack Surface	2
1.2.2	Automated Driver Extraction and Isolation	3
1.2.3	Program Analysis Techniques	3
1.3	Contributions	4
1.4	Thesis Organization	5
2	Background	7
2.1	The AV1 Video Codec	7
2.2	Linux Device Driver Model	8
2.3	The V4L2 Media Subsystem	9
2.3.1	V4L2 Framework	9
2.3.2	Videobuf2 Framework	9
2.3.3	Memory-to-Memory Framework	11
2.3.4	Subsystem Layering and Interconnection	11
2.4	Stateless Video Decoding	12
2.4.1	The Request API	12
2.4.2	AV1 Decoding on the RK3588	13
2.4.3	Post-Processing and Film Grain	13
2.5	Kernel Polymorphism and Analysis Challenges	14
2.6	Static Analysis with CodeQL	15
2.6.1	Fundamentals and AST Interactions	15
2.6.2	Call Graph Resolution and Recursion	16
2.6.3	Variable Access and Data Flow Analysis	16
2.7	Dynamic Kernel Instrumentation	17
2.7.1	eBPF Architecture	17
2.7.2	<code>strace</code> , <code>ftrace</code> , and <code>kprobes</code>	17
2.7.3	The <code>bpftrace</code> Tracing Language	18
3	Implementation	19
3.1	Establishing the Research Environment	20
3.1.1	Hardware Initialization and Power Constraints	20
3.1.2	Kernel Integration: OpenCCA and AV1 Support	20
3.1.3	Kernel Configuration and Instrumentation	21

3.1.4	Bootloader Configuration and Serial Console Debugging . . .	21
3.1.5	GStreamer Build and Configuration	21
3.2	Manual Analysis	22
3.3	Static Analysis with CodeQL	23
3.3.1	Database Generation and Scoping	23
3.3.2	Developing the Hantro QL Library	24
3.3.3	Direct and Indirect Call Invocations	24
3.3.4	Filtering Indirect Calls	26
3.4	Shared Struct Access Analysis	27
3.4.1	Syntactic Field Access	28
3.4.2	Dataflow Taint Tracking	28
3.5	Dynamic Verification using bpfttrace	29
3.5.1	Handling Inlined Functions	29
3.5.2	Automated Trace Generation	30
4	Results	31
4.1	Analysis Scope and Target ioctls	31
4.2	Visualization of the Call Graphs	33
4.3	Methodology Validation	35
4.4	Kernel Dependencies for AV1 Decoding	37
4.5	Hardware-Specific ops Struct Implementations	39
4.6	Dynamic Validation via bpfttrace	42
4.6.1	Confirmed Execution Paths	42
4.6.2	Statically Reachable but Not Executed	43
4.6.3	Untraceable Functions	44
4.7	Shared Struct Access Dependencies	44
4.7.1	Syntactic Field Access	45
4.7.2	Dataflow Taint Tracking	45
4.7.3	Propagation Gap of Taint Tracking	46
5	Conclusion	47
5.1	Summary of Contributions	47
5.2	Methodology Evaluation	47
5.3	Limitations	48
5.4	Future Work	50
5.5	Closing Remarks	51
	List of Figures	53
	List of Tables	54
	List of Listings	55
	Bibliography	56

A	Research Artifacts	63
B	Environment Configuration	65
B.1	Kernel Config File	65
B.2	Clangd Configuration File (.clangd)	66
C	CodeQL Queries	67
D	Discrepancy Taxonomy	69
E	Cross-Module Call Graph	71
F	ops Struct Dynamic Results	77
G	Shared Structs	79
H	Static vs. Dynamic Coverage	81

1 Introduction

This chapter establishes the foundational context and rationale for the thesis. Section 1.1 outlines the inherent security risks associated with monolithic kernel device drivers and the industry’s increasing reliance on complex hardware accelerators. Building upon this problem statement, Section 1.2 evaluates existing literature concerning hardware isolation and program analysis. It demonstrates why current mitigation strategies fall short for stateless multimedia pipelines, creating the necessary interconnection that justifies our driver extraction methodology. Then, Section 1.3 states the contribution that this thesis provides. Finally, Section 1.4 functions as an overview of how this thesis is organized.

1.1 Motivation

In the modern Linux ecosystem, device drivers act as a bridge between physical hardware and high-level software abstractions [CRKH05]. While modularity enables dynamic loading without system reboots, it introduces a hidden security vulnerability: drivers execute with full kernel privileges. Consequently, a single vulnerability within a driver can enable an attacker to compromise the entire host system, bypassing all operating system boundaries [Sil26].

The expansion of mobile capabilities into resource-heavy domains like augmented reality, 4K media processing, and on-device artificial intelligence amplifies this architectural risk [WLW⁺17]. As traditional CPUs encounter physical constraints in clock speed and thermal management due to the failure of Dennard scaling, modern hardware design has shifted toward heterogeneous computing architectures and Hardware accelerators [EBSA⁺12]. Hardware accelerators compute specific tasks with extreme efficiency. However, they require complex, often proprietary software stacks and privileged device drivers with Direct Memory Access (DMA) to system memory (Section 2.2). This complexity widens the system attack surface.

The motivation of this thesis is to address such vulnerabilities: we aim to understand the intricate dependencies of these drivers and investigate how to run the driver independently of the Linux kernel.

1.2 Related Work

This section reviews existing literature on hardware multimedia acceleration, operating system-level driver isolation, and program analysis for reverse engineering. By analyzing the results of these prior works, we highlight their limitations when applied to complex, stateless multimedia pipelines, thereby establishing the relevance of our extraction approach.

1.2.1 Hardware Acceleration and the Attack Surface

Hardware acceleration is a requirement for modern video coding standards such as HEVC/VVC and AV1 (Section 2.1) due to their computational complexity [BWC⁺21]. While historical vulnerability research has focused on central OS components or GPU drivers, modern mobile devices integrate dozens of less-audited, privileged accelerators (e.g., VPUs, NPUs, and DSPs) that remain accessible from untrusted user-space contexts. Maar et al. [MDS⁺24] mapped this attack surface across major Original Equipment Manufacturers (OEM), resulting in the discovery that media and accelerator drivers represent a massive, easily reachable target for attackers, as illustrated in Figure 1.1.

Category	Device Driver's Entry	Module	Accessibility per OEM Vendors						
			Samsung	Xiaomi	Asus	Realme	OnePlus	Oppo	Vivo
AI	/dev/apuext	apusys.ko	2	12		12		29	40
DSP	/dev/adsprpc-smd	frpc-adsprpc.ko	48	52	83	56	71	43	40
DSP	/dev/fastrpc-[acs]dsp	frpc-adsprpc.ko		10					
NPU	/dev/vertex	npv.ko	38						
AI	/dev/apusys	apusys.ko		4					
Audio	/dev/xlog	xlogchar.ko		60					
GPU Extension	/proc/ged	ged.ko	14	38	17	38	29	57	60
Monitor	/proc/perfmgr	mtk_perf_ioctl.ko	7	38	17	38	29	57	60
JPEG	/proc/mtk_jpeg	jpeg-driver.ko	2	25	17		14	29	40
Memory	/proc/secmem	trusted_mem.ko	12	25					
Monitor	/proc/mi_log	mi_log.ko		21					
Camera	/proc/camera	camera.ko	10						

Figure 1.1: Accessible kernel attack surface from untrusted security contexts, detailing the percentage of Android OEM devices permitting user-space access to hardware drivers (adapted from [MDS⁺24]).

This accelerator attack surface is frequently demonstrated in the wild. For example, recent vulnerability research by Google Project Zero resulted in a zero-click exploit chain on the Pixel 9 [JSF26]. Similarly, vulnerabilities in Qualcomm’s video accelerator drivers (CVE-2022-22087 [Nat22]) and MediaTek’s DSPs have allowed local privilege escalation [Mak21]. Moreover, the USENIX Security Paper aptly called “The Most Dangerous Codec in the World: Finding and Exploiting Vulnerabilities in H.264 Decoders” [VCS23] directly shows many inherent vulnerabilities of stateless decoding drivers. These results underscore why completely extracting

and isolating hardware drivers from the kernel context is the objective of this thesis.

1.2.2 Automated Driver Extraction and Isolation

To mitigate the risk of drivers compromising the operating system, modern secure systems are shifting toward hardware-software isolation [HHE⁺24].

A historical strategy involves over-approximating the hardware and software environments to run drivers inside emulators. For instance, the Avatar framework forwards I/O operations between a physical device and a virtual machine to facilitate dynamic security analysis [ZBFB14]. While this enables fuzzing, full emulation introduces severe latency, making it unsuitable for real-time multimedia decoding.

More recent approaches aim to isolate drivers using distinct architectural boundaries. While frameworks like *LXDs* (Lightweight Execution Domains) [NBJ⁺19] and *KSplit* [HND⁺22] attempt to retrofit these boundaries within monolithic kernels by executing drivers in separate address spaces and generating synchronization glue-code, clean-slate architectures provide a more robust foundation. For example, recent research into Neural Processing Units (NPUs) achieved viable micro-kernel decomposition and IOMMU-backed sandboxing with minimal performance overhead [FFD⁺25].

However, decomposing stateless multimedia drivers remains challenging. Unlike network cards or simple accelerators, codec drivers depend on high-bandwidth memory translations and complex `ioctl` handshakes between user-space and the kernel (Section 2.3). This thesis addresses these challenges by systematically analyzing the kernel-driver interactions of a stateless AV1 decoder using multiple complementary methods, precisely characterizing the boundary a driver must expose to operate outside its native kernel environment.

1.2.3 Program Analysis Techniques

Analyzing and extracting monolithic kernel drivers requires navigating complex code and internal subsystem dependencies. Understanding these boundaries necessitates static and dynamic program analysis [Ern03].

Static analysis tools excel at identifying structural patterns across codebases. Systems like SHARD utilize fine-grained static analysis, resulting in context-aware kernel hardening by restricting control flows [AAFX21]. Recently, semantic query engines like CodeQL (Section 2.6) have been applied to the Linux kernel to trace complex, cross-entry data flows [W⁺25]. However, static analysis struggles to resolve dy-

dynamic function pointer bindings, such as those used in the V4L2 `ops` struct dispatch pattern; Section 3.3.3 addresses this limitation using CodeQL directly.

Dynamic analysis complements static analysis by observing actual runtime behavior. The modern kernel tracing infrastructure built on eBPF and high-level frontends such as `bpftrace` enables safe, low-overhead introspection of live kernel data structures without modifying kernel source (Section 2.7), making it well-suited for confirming which statically predicted paths actually execute. This approach isolates a precise execution path necessary to run the hardware driver autonomously.

1.3 Contributions

The objective of this thesis is to analyze the interactions between the Hantro AV1 decoding driver and the kernel, define all interfaces used, and list all components a driver needs to run independently of the Linux kernel while still providing its full functionality to the kernel. To that end, this thesis delivers the following contributions:

- **Extensible CodeQL Static Analysis Library:** A CodeQL library that computes the transitive closure of direct and indirect calls across the Linux kernel media subsystem (Section 3.3). By resolving the concrete `ops` struct implementations active during AV1 decoding, the library reduces static over-approximation by excluding unrelated codec and memory backend implementations (Section 4.5). Complementary queries map cross-boundary struct field accesses, precisely characterizing the data structures the driver exchanges with the kernel during `ioctl` execution.
- **Automated Dynamic Verification Pipeline:** A Python pipeline that parses CodeQL output and generates `bpftrace` probe scripts for all traceable call targets (Section 3.5). Executing these probes against a AV1 decode session produces per-function invocation counts, confirming active code paths, verifying the `ops` struct implementations identified by static analysis, and distinguishing unreachable functions from static over-approximations.
- **Empirical Validation:** Validating the static analysis library against a manually traced ground truth of the `REQBUFS ioctl` call. By cross-referencing static predictions with dynamic tracing data, we isolate and reconcile structural discrepancies, delivering a verified, precise mapping of the kernel-driver boundary.

1.4 Thesis Organization

Following this introduction, the remainder of this thesis is structured into four chapters, moving from theory to implementation, evaluation, and conclusions:

- **Chapter 2:** Background provides the technical landscape required to understand the system environment. It explores the AV1 codec format, details the Linux device driver model, and the Video4Linux2 (V4L2) media subsystem. Finally, it introduces stateless multimedia decoding and abstract syntax tree (AST) interactions within CodeQL, alongside dynamic tracing using eBPF and `bpftime`.
- **Chapter 3:** Implementation details the methodology we executed in this research. It documents the environmental initialization, kernel configurations, and user space setup. It then outlines the implementation of our analysis: manual call graph tracing utilizing Language Server Protocols (LSP) and static analysis using our Hantro QL library for direct, indirect, and taint tracking. The chapter concludes by detailing the runtime validation framework built using `bpftime`.
- **Chapter 4:** Results delivers an evaluation and analysis of our findings. It presents the resolved target driver call graphs and evaluates the call graphs to find subsystem dependencies. We show the results of the dynamic analysis to verify the static framework. Finally, we present the shared structs yielded by our cross-boundary analysis.
- **Chapter 5:** Conclusion summarizes the insights produced by this work, outlines the boundaries and limitations encountered by our methodology, and proposes directions for future work regarding automated driver isolation and translation.
- **Appendices:** Provide supplementary technical materials, including the research artifact repository, configuration files, CodeQL queries, and the complete datasets for our call graphs, shared structs, and dynamic coverage results.

2 Background

This chapter provides background concepts that are essential to understanding the subsequent chapters. Section 2.1 introduces the AV1 codec. Section 2.2 explains the structure of a linux device driver. Section 2.3 provides an overview of the V4L2 subsystem. Section 2.4 elaborates on stateless decoding. Section 2.5 shows ways in which the kernel achieves polymorphism with C. Section 2.6 introduces CodeQL as a static analysis language. Finally, Section 2.7 shows the dynamic analysis tools used in this thesis.

2.1 The AV1 Video Codec

AV1 is a royalty-free, open-source video codec developed by the Alliance for Open Media (AOMedia) in 2018 that delivers approximately 30% better compression efficiency than its predecessor, H.265 (HEVC) [CMHea20]. Due to these efficiency gains, it has become the state-of-the-art standard for internet video distribution and is widely deployed by major streaming platforms including YouTube, Netflix, and Amazon Prime Video [Bit25b].

Video codecs work by encoding raw pixel data into a compact binary representation called a *bitstream*, which a decoder interprets to reconstruct the original frames [Ric04]. Achieving AV1’s compression efficiency requires computationally intensive algorithms that exceed the real-time processing capabilities of general-purpose CPUs at high resolutions [CMHea20]. Practical AV1 deployment, therefore, depends on dedicated hardware accelerators, such as the Video Processing Unit (VPU) integrated into the Rockchip RK3588 SoC used in this thesis. These implement the decode algorithms directly in silicon [Rad26b]. As shown in Table 2.1, AV1 adoption is growing rapidly as such hardware blocks become standard features in modern SoCs.

Table 2.1: Comparison of Video Codec Market Share (Bitmovin Reports 8 & 9)

Report Edition	H.264 (AVC)	H.265 (HEVC)	AV1
Report 8 (2024/25) [Bit25a]	79%	49%	13%
Report 9 (2025/26) [Bit25b]	65%	41%	16%

2.2 Linux Device Driver Model

The Linux kernel abstracts hardware devices to isolate architecture-specific code from core system operations [CRKH05]. On System-on-Chip (SoC) platforms like the Rockchip RK3588, the Device Tree specification describes which hardware peripherals are present at boot, allowing the kernel to bind each peripheral to the correct driver [The24k]. During initialization, the driver’s `probe()` routine claims the hardware resources the peripheral requires—register addresses, clocks, and interrupt lines [The24p]—using the `devres` API, which automatically releases them if initialization fails or the driver is unloaded [The24d]. This tight coupling between driver code and kernel infrastructure means a driver cannot be executed in isolation: any attempt to run driver logic outside the kernel requires emulating the kernel services it depends on, which is the central challenge this thesis addresses.

User-space applications communicate with kernel drivers through the Input/Output Control (`ioctl`) system call [Lin25]. Unlike `read` and `write`, which transfer raw byte streams, `ioctl` acts as a structured command channel: an application issues a command code paired with a data structure, which the kernel dispatches to the appropriate driver handler. Within the Video4Linux2 (V4L2) framework [The24s], `ioctl` calls are the primary mechanism through which applications allocate decode buffers, submit compressed video frames, and retrieve decoded output, making them the natural entry points for the analysis in this thesis.

Kernel Infrastructure Dependencies Hardware drivers operate within an expansive ecosystem of internal kernel services. Safely executing a driver inside an isolated microkernel environment requires identifying these interfaces:

- **Memory Management & DMA:** Core APIs (such as `dma_alloc_coherent()`) handle physical memory layouts, IOMMU translations, and cache coherency protocols essential for hardware access [The24e].
- **Dynamic Allocation (Slab):** Runtime heap allocations (`kzalloc()`/`kfree()` using `slab.h`) are used to build dynamic, per-frame data structures during active decoding cycles [The24m].
- **Synchronization Primitives:** Drivers manage multi-threaded concurrency using internal lockers, such as spinlocks, mutexes, and completion signals [CRKH05].
- **Interrupts & Workqueues:** `request_irq()` registers low-level interrupt handlers, while the workqueue sub-framework (`workqueue.h`) defers non-critical bottom-half completion routines [HM10].

- **Clock and Power Management:** SoC peripherals depend on complex clock trees and power domains governed by the Common Clock Framework and run-time PM (`clk_prepare_enable()`, `pm_runtime_get_sync()`) [The24b, The24q].
- **Logging & Utilities:** Basic kernel helpers (`printk.h`, `string.h`) are invoked continuously for diagnostics (`dev_err`) and memory operations (`memcpy/memset`) [The24j].

This can be compared to the results in Section 4.4 where the subsystems are categorized based on the number of `ioctl`s that reach them for AV1 decoding.

2.3 The V4L2 Media Subsystem

The Video4Linux2 (V4L2) subsystem is the Linux kernel framework for video capture, output, and hardware codec operations [The24s]. It exposes hardware video devices as standard character device files, allowing user-space applications to interact with diverse hardware through a unified API without requiring knowledge of the underlying hardware implementation.

2.3.1 V4L2 Framework

V4L2 functions as the main entry point for multimedia pipelines, exposing video devices through standard file system paths (`/dev/videoX`). User-space applications interact with these devices via standard POSIX system calls: `open()`, `close()`, `ioctl()`, and `mmap()` [CRKH05].

The core of the V4L2 interface is `ioctl()`-based: applications first query device capabilities via `VIDIOC_QUERYCAP` to determine whether a device supports video capture, output, or memory-to-memory operations. Format negotiation is then performed through `VIDIOC_G_FMT` and `VIDIOC_S_FMT`, which configure pixel format, resolution, and color space for both the compressed input stream and the decoded output (including raw NV12 frames) [The24s]. Additionally, the V4L2 Event API (`v4l2-event.c`) allows user-space applications to subscribe to asynchronous device state changes, such as source format modifications or decoder error conditions [The24g].

2.3.2 Videobuf2 Framework

The Videobuf2 (VB2) framework manages video buffer allocation, lifecycle tracking, and streaming I/O within V4L2 [The24t]. It governs buffer ownership as data moves between user-space and the hardware: buffers begin under user-space control and transition to a queued state within the framework. The driver then claims these

buffers upon submitting them to the hardware decoder, finally returning them to user-space upon completion.

During initialization, drivers assign a `vb2_mem_ops` struct to select one of three memory allocators: `vb2-dma-contig` for physically contiguous memory via the Contiguous Memory Allocator (CMA) [The24c]; `vb2-dma-sg`, which allocates discontinuous pages and maps them for DMA using the Input/Output Memory Management Unit (IOMMU); and `vb2-vmalloc`, which allocates memory that is contiguous only in the virtual address space. `vb2-vmalloc` lacks the physical contiguity required by hardware DMA controllers, making it suitable only for CPU-bound software processing [The24m]. The Hantro VPU requires physically contiguous buffers because its DMA engine accesses frame data via linear address increments and cannot traverse a scatter-gather list [RV24]. Consequently, the driver explicitly selects `vb2-dma-contig`, rendering the other two backends inactive for this hardware configuration.

While the memory allocators dictate how the kernel provisions the storage, VB2 separately defines three buffer allocation modes to govern how subsystems exchange ownership of that storage. These modes do not change the allocation backend itself; instead, they establish distinct execution and accessibility models for the physical memory holding compressed input and decoded output frames:

- **Memory-Mapped Buffers (MMAP):** The kernel allocates physically contiguous buffers via the DMA-contig allocator and maps them into user-space via `mmap()` [Lin26a]. Both user-space and the hardware peripheral access the same physical pages directly, avoiding expensive data copies. This mode suits self-contained pipelines where user-space acts as both the producer and consumer of frame data. As detailed in Section 4.6.2, the AV1 decoding pipeline analyzed in this thesis utilizes this mode.
- **User Pointer Buffers (USERPTR):** User-space allocates the buffer directly and passes a pointer to the kernel, which pins the corresponding physical pages for hardware DMA access [The24o]. This grants user-space full control over buffer placement but imposes strict alignment and contiguity requirements that the application must satisfy.
- **DMA Buffers (DMABUF):** One kernel subsystem allocates a buffer and exports it to other subsystems via a file descriptor [The24e]. Each subsystem imports this file descriptor and maps the shared physical memory region for independent hardware access, eliminating intermediate copies. This shared-buffer model is the established standard for high-throughput production multimedia pipelines [Par25], where a VPU decodes directly into a DMABUF that a display engine or GPU subsequently imports and composites.

By centralizing buffer allocation and ownership tracking, VB2 enforces safe concurrent access among user-space applications, the monolithic kernel, and DMA-capable

hardware. This management mitigates the risk of memory corruption, such as use-after-free and double-mapping vulnerabilities.

2.3.3 Memory-to-Memory Framework

Hardware video decoders consume compressed bitstream data and produce raw, displayable video frames. The V4L2 Memory-to-Memory (M2M) framework models this by maintaining two buffer queues per decode session [The24r]: the **OUTPUT** queue carries compressed bitstream data from user-space to the hardware, and the **CAPTURE** queue holds the resulting decoded frames in a raw pixel format ready for display or further processing. The naming follows the hardware’s perspective where **OUTPUT** is what the CPU sends out to the device, and **CAPTURE** is what the CPU retrieves from it. Despite its name, M2M is a V4L2 utility rather than a separate subsystem layer.

The framework’s job scheduler isolates concurrent decode sessions through per-context state (`struct v4l2_m2m_ctx`). When buffers are available on both queues, it triggers the driver’s `device_run()` callback, which programs the hardware registers and initiates the decode operation.

2.3.4 Subsystem Layering and Interconnection

A Linux hardware decoding pipeline routes a buffer through several kernel layers in a fixed order, illustrated in Figure 2.2. A more complete view of these subsystem interactions is given later in Figure 4.5 (Section 4.2).

When user-space submits a buffer, the V4L2 core validates the `ioctl` parameters and, through the M2M framework, a V4L2 utility that associates the operation with the correct `v4l2_m2m_ctx` decode session, forwards the request to `Videobuf2`. `VB2` advances the buffer through its ownership lifecycle, ensures the underlying physical memory pages are pinned and safe for DMA access, and then enqueues the buffer to the driver once both the **OUTPUT** and **CAPTURE** queues hold ready buffers. The driver programs the VPU’s registers and triggers the decode operation on the hardware. Once the hardware interrupt signals completion, the driver calls `vb2_buffer_done()` directly, handing the buffer back to `VB2` for return to user-space.

Understanding this layered sequence of boundary crossings between V4L2, M2M, `VB2`, and the hardware driver is fundamental to identifying the minimal set of kernel abstractions that must be replicated for isolated driver execution.

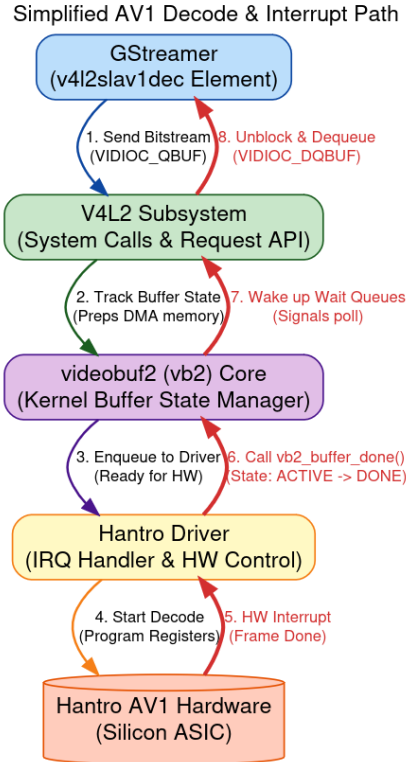


Figure 2.2: Dataflow during a hardware decode request.

2.4 Stateless Video Decoding

Modern SoC video processing units (VPUs) increasingly utilize a stateless architecture, shifting the burden of bitstream parsing from hardware firmware to user-space software [Duf20]. Traditional *stateful* codecs embed a microcontroller that parses the bitstream and tracks reference frames internally, requiring user-space only to feed raw compressed data [The24n]. A *stateless* hardware decoder, by contrast, acts purely as an execution engine with no internal state. User-space must parse the bitstream itself, extracting per-frame metadata, and supply it alongside each compressed frame as kernel-defined control structures. The hardware then processes exactly one frame per submission, returning raw decoded pixel data without retaining any state between frames [Duf20].

2.4.1 The Request API

Traditional V4L2 interfaces were designed for stateful devices operating on continuous streams and lack a mechanism to atomically bind per-frame metadata to a

specific buffer. For stateless hardware decoders, which require explicit codec parameters for every individual frame, this is insufficient. The Linux kernel’s Request API [The24l] addresses this by allowing user-space to bundle V4L2 control parameters and a data buffer into a single atomic execution unit called a request. The kernel guarantees that the hardware receives both the buffer and its associated metadata together before initiating the decode.

For AV1 specifically, this metadata is extensive: user-space must supply a collection of AV1 specific control structures (such as `v4l2_ctrl_av1_frame`) [The26], covering quantization, filtering, and motion compensation parameters for every frame. The driver maps these directly into VPU register writes. This mechanism produces a distinct `ioctl` sequence for every decoded frame, interleaving buffer management and control submission across the media controller layers. Accurately tracing this sequence is a central objective of the analysis conducted in Chapter 3.

2.4.2 AV1 Decoding on the RK3588

The Rockchip RK3588 is a high-performance ARM System-on-Chip (SoC) targeting single-board computers, embedded AI, and media applications [Roc22]. The Radxa Rock 5B is a single-board computer built around the RK3588, providing a development platform with full Linux support and hardware video acceleration [Rad26b]. The Rockchip RK3588 SoC integrates a dedicated Video Processing Unit (VPU) built on VeriSilicon’s Hantro hardware IP [Ver24], a widely licensed decoder core found across multiple SoC vendors. Within the Linux kernel, this hardware block is managed by the Hantro driver located in `drivers/media/platform/verisilicon/`, which serves as the bridge between the V4L2 stateless decoder API and the VPU register interface [RV24], mapping per-frame metadata received via the Request API directly into hardware register writes.

2.4.3 Post-Processing and Film Grain

VeriSilicon pairs the core decode engine with an optional hardware post-processor that performs color space conversion and scaling on decoded output [Ver24]; the Hantro driver bypasses it entirely if the requested output format matches the decoder’s native tiled memory layout [RV24], a behavior confirmed dynamically in this thesis in Section 4.6.2.

Film grain is the fine, random texture naturally present in video captured on film, which many sources retain for aesthetic reasons [CMHea20]. However, it compresses poorly because grain is essentially noise. AV1 avoids this cost by having the encoder strip the grain before compression and embed a small set of parameters describing its appearance in the bitstream, leaving the decoder to regenerate a visually similar

pattern and overlay it after decoding. In the Hantro driver, the hardware registers needed for this regeneration are programmed on every frame, while the CPU-side computation that generates the grain pattern only runs when the bitstream signals that film grain is present.

2.5 Kernel Polymorphism and Analysis Challenges

The Linux kernel is written in C, which has no native support for polymorphism, the ability of a single interface to dispatch to different concrete implementations depending on the context. The kernel implements this through operations structs (**ops** structs): structs whose fields are function pointers [CRKH05]. A subsystem defines the struct as an interface contract. Each driver provides its own instance, with function pointer fields assigned to its concrete implementations at compile time. At runtime, during driver initialization, the driver registers its instance with the subsystem. From that point onward, the subsystem invokes the driver's functions through the stored pointers without any knowledge of the underlying hardware.

Listing 2.3 illustrates this with Videobuf2's `struct vb2_ops` [The24t]. VB2 defines the buffer lifecycle interface; the Hantro driver fulfills it by binding its own functions during initialization. When VB2 needs to prepare or queue a buffer, it invokes the callback through the function pointer without any knowledge of the underlying hardware.

```

struct vb2_ops {
    int (*buf_prepare)(struct vb2_buffer *vb);
    void (*buf_queue) (struct vb2_buffer *vb);
    // ...
};

static const struct vb2_ops hantro_qops = {
    .buf_prepare = hantro_buf_prepare,
    .buf_queue = hantro_buf_queue,
    // ...
};

```

Listing 2.3: Polymorphism via function pointer structures in Videobuf2.

This pattern keeps each subsystem independent of specific hardware, while all hardware-specific logic stays contained within the driver, but it presents a fundamental challenge for static analysis [MRR04]. A call such as: `ops->buf_prepare(vb)` carries no information at the call site about which concrete function executes. Standard tools such as `grep` (a text based search tool) or go-to-definition navigation completely miss the driver-specific implementation. The actual binding occurs at runtime during device initialization, when the driver registers `&hantro_qops` into the active `vb2_queue` struct.

This is precisely where CodeQL’s `ClassAggregateLiteral` node attaches (Section 2.6.1), enabling the static resolution of function pointer assignments described in Section 3.3.3. As static resolution remains ambiguous, dynamic analysis captures the concrete symbol resolutions during live execution on the hardware platform (Section 3.5).

2.6 Static Analysis with CodeQL

CodeQL compiles source code into a queryable database, allowing researchers to express complex flow patterns as declarative queries rather than writing custom analysis tools from scratch [Git26b]. For C/C++ projects like the Linux kernel, CodeQL parses the source into an Abstract Syntax Tree (AST), which is a tree representation of the code’s syntactic structure where each node corresponds to a language construct, such as a function, a variable access, or a function call. CodeQL exposes these nodes through a class hierarchy that mirrors C/C++ language constructs, allowing queries to select and filter them by type.

2.6.1 Fundamentals and AST Interactions

A CodeQL query is built upon logical functions called *predicates*, which express conditions over AST nodes and can be composed to build more complex queries [Git26a]. For example, the custom predicate `isDriverFile(File f)` returns true if the file path contains `verisilicon/`, allowing all subsequent queries to restrict their scope to the Hantro driver without repeating that condition.

To trace execution patterns within the kernel, queries traverse various AST node types, some of them are listed here:

- **Function and Call:** Represent function declarations and direct function invocations, respectively.
- **ExprCall and PointerFieldAccess:** Model indirect invocations through a struct field function pointer (such as `ops->queue_setup(...)`).
- **FunctionAccess:** Represents a function being referenced by name and passed as an argument to another function, without being called directly at that point.
- **ClassAggregateLiteral:** Models the static initialization of a C struct. This is critical for resolving polymorphic operations structures (Section 2.5), as it links an abstract `Field` of a `Struct` to the concrete function assigned to it at compile time [Git26b].

Queries use the `exists()` formula to introduce temporary local variables and establish relationships between AST nodes. For example, asserting that there exists a `ClassAggregateLiteral` that assigns a specific function to a given struct field.

2.6.2 Call Graph Resolution and Recursion

Constructing an accurate call graph requires mapping both direct and resolved indirect calls. Once we define a base `calls(caller, callee)` predicate establishing a single caller-to-callee relationship (Listing 3.3), CodeQL can extend it using *transitive closure* operators: `+` finds all nodes reachable in one or more steps, while `*` additionally includes the starting node itself (zero or more steps) [Git26a]. Applying `calls*(entryPoint, reachable)` therefore yields every function transitively reachable from a given `ioctl` entry point, no matter how deeply nested. A full traversal that would be impractical to perform manually across the kernel source.

To manage the volume of results generated by recursive queries, CodeQL results can be filtered or aggregated to improve readability. We mainly use filtering with predicates such as `isSubsystemFile(caller)` to find only calls where a caller is in the subsystem (Section 3.3.2).

2.6.3 Variable Access and Data Flow Analysis

Beyond control flow, identifying which kernel structs the driver reads or writes during execution is necessary to determine the full set of kernel dependencies. CodeQL’s `VariableAccess` nodes track every syntactic reference to a variable or struct field in the AST [Git26b], yielding a complete set of accessed structures across the codebase. This approach is conservative: it captures every textual reference regardless of whether that code path executes at runtime, making it complete but imprecise [MS18]. For example, error handling paths, debug routines, or codec configurations irrelevant to AV1 decoding are all syntactically reachable but are never triggered during a standard decode workload.

CodeQL’s data-flow and taint-tracking libraries model semantic value propagation from a defined *source* to a *sink*, traversing assignments, pointer arithmetic, and cross-function boundaries [Git26a]. In principle, this allows taint tracking to isolate only the structs that a value provably reaches during execution, trading the conservative completeness of syntactic access for higher precision. In practice, this precision depends on the analysis correctly modeling every relevant flow step; unresolved function pointers, struct field aliasing, or unmodeled flow patterns can cause the tracked taint to break prematurely, silently omitting structs that are genuinely

accessed. Section 3.4 applies both approaches to the Hantro driver and compares their results directly.

2.7 Dynamic Kernel Instrumentation

While static analysis maps structural relationships, dynamic instrumentation observes runtime behavior, validates execution paths, and captures live state without kernel modification or recompilation [Gre19].

2.7.1 eBPF Architecture

The Extended Berkeley Packet Filter (eBPF) is an in-kernel virtual machine for executing dynamic, sandboxed programs [The24f]. Evolving from network filtering, modern eBPF provides general-purpose infrastructure for tracing, observability, and security.

Before execution, source code is compiled into eBPF bytecode. The eBPF Verifier statically analyzes this bytecode to prevent kernel crashes, rejecting infinite loops and unsafe memory accesses [Tha25]. Once verified, a Just-In-Time (JIT) compiler translates it into native machine instructions [The24f]. This verification and compilation pipeline ensures safe, near-native execution.

2.7.2 `strace`, `ftrace`, and `kprobes`

The `strace` user-space utility intercepts and logs process system calls [Str24]. Filtering its output for `ioctl` commands during AV1 decoding isolates the V4L2 request sequence, exposing the initial call graph entry points traced in Chapter 3.

The `ftrace` framework provides dynamic function-level instrumentation [Ros24]. With `CONFIG_FTRACE` enabled, the compiler inserts a no-op at every kernel function entry. Probes dynamically patch these no-ops to execute callbacks, logging calls, arguments, and returns with negligible overhead when inactive, bypassing the need for kernel recompilation.

The `kprobes` subsystem enables dynamic probe attachment [The24i, MPK⁺06]. Registered by symbol name, a `kprobe` intercepts execution to run a callback. With `CONFIG_FTRACE`, `kprobes` efficiently reuse `ftrace` no-ops; without it, they fall back to invasive breakpoint instructions. In `bpftime`, `kprobe:function_name` declarations automatically register probes and attach eBPF callbacks, serving as this thesis’s foundational probing layer.

Both trace mechanisms resolve targets via symbol names, necessitating kernel symbol-to-address mappings. Enabling `CONFIG_KALLSYMS_ALL` extends this mapping to un-exported internal functions [Lin26c], exposing the driver’s full internal call graph to tracing utilities.

Crucially, neither tool can instrument inlined functions, since compilers embed their bodies directly at call sites and remove the corresponding symbol entry point. Section 3.5.1 discusses how we address and ultimately accept this limitation.

2.7.3 The bpftrace Tracing Language

The **bpftrace** tracing language provides a high-level syntax over eBPF [Gre19], abstracting bytecode generation. It simplifies `kprobe` declaration, statistics aggregation, and runtime data structure access.

BPF Type Format (BTF) enables struct-level introspection [The24a]. BTF embeds C type information (layouts, names, offsets) into the kernel binary, letting **bpftrace** resolve fields by name dynamically. Without BTF, field access requires manual, fragile offset lookups via **pahole** - a tool that reads debug information [CdM23], which break upon recompilation. BTF eliminates this overhead, providing robust runtime type resolution.

This introspection solves the kernel polymorphism challenge (Section 2.5). While static analysis fails to resolve pointers like `ops->buf_queue`, **bpftrace** can intercept callers, dereference the struct, and extract the concrete target symbol via `ksym()`, resolving ambiguities at runtime (Listing 2.4).

```
kprobe:vb2_core_qbuf
{
    $q = (struct vb2_queue *)arg0;
    printf("Active queue_setup implementation: %s\n",
        ksym($q->ops->queue_setup));
}
```

Listing 2.4: Example bpftrace script resolving a dynamic function pointer target.

3 Implementation

This chapter presents a three-stage methodology for analyzing the kernel-driver boundary of the Hantro VPU driver during AV1 decoding on the Rockchip RK3588 SoC. The three stages are not independent pipelines; each stage directly informs the next:

1. **Manual analysis** (Section 3.2): We trace the complete call graph for `VIDIOC_REQBUFS` by hand. This establishes a ground truth and provides the intuition needed to design the static analysis library. While precise, manual tracing does not scale beyond a single code path.
2. **Static analysis** (Section 3.3): Using the manual ground truth, we build a CodeQL library that resolves direct and indirect calls through `ops` structs. This scales to the full driver but over-approximates by including paths that never execute [MS18, NNH99]. An additional query (Section 3.4) maps cross-boundary kernel struct accesses.
3. **Dynamic analysis** (Section 3.5): This stage confirms execution during an active decoding session, though it misses edge cases that are not triggered by the test pipeline. It validates static assumptions by identifying which `ops` implementations actually run, providing frequency data to filter out unreachable paths.

Together, the three stages achieve completeness (by statically capturing all reachable paths) and accuracy (by dynamically verifying active paths) [Ern03]. Chapter 4 presents the results.

Once we validated the static analysis library against the manual ground truth, it operates independently: the manual trace is not required for subsequent analyses of other entry points or drivers. The `bpfftrace` automation further reduces manual effort by generating probes directly from static analysis output, leaving only the initial query execution and filter population as manual steps.

Executing the analysis required a fully functional hardware and software environment. Section 3.1 details the deployment of this research platform, outlining how we resolve non-trivial boot challenges and integrate a mainline-aligned kernel with hardware-accelerated AV1 decoding capabilities.

3.1 Establishing the Research Environment

The Radxa Rock 5B, featuring the Rockchip RK3588 SoC, served as the hardware target. We selected this platform for three reasons: its Hantro VPU provides hardware-accelerated AV1 stateless decoding [RV24]; the RK3588 has seen substantial mainline Linux integration [BS25]; and the Rock 5B is low-cost, widely available, and well-supported by an active community [Rad26b]. These advantages came with non-trivial integration challenges, detailed below.

3.1.1 Hardware Initialization and Power Constraints

The Radxa Rock 5B exhibits strict power requirements [Rad26b]. Testing revealed that during the transition from U-Boot to kernel initialization, the RK3588 SoC’s power spikes frequently exceed the current limits of some Power Delivery (PD) adapters and integrated host ports. As summarized in Table 3.1, stable operation depended on the power source’s ability to negotiate voltage rails and sustain peak current draws. The Aukey QC 3.0 adapter provided stable operation for all experiments in this thesis.

Table 3.1: Power Supply Compatibility Observations

Power Source	Type / Rating	Result
Aukey QC 3.0 (USB-A)	18W (up to 12V/1.5A)	Stable
Digitec GaN (USB-C)	65W (Multi-profile)	Stable
ThinkPad TB4 Port	Thunderbolt 4 (Host)	Stable
Apple USB-C Adapter	20W PD	No Power
ThinkPad USB-A 3.2	Integrated PC Port	Boot Loop
ThinkPad Slim Charger	65W (Fixed PD)	Failed

3.1.2 Kernel Integration: OpenCCA and AV1 Support

To enable realistic analysis of modern driver architectures, we replaced the vendor OS (utilizing a downstream 6.1 kernel) with the OpenCCA kernel (version 6.12)[BS25]. This kernel provides greater adherence to the mainline Linux architecture, which is essential for accurate driver behavior analysis.

The build process leveraged the vendor-provided Board Support Package (BSP) toolchain[Rad26a]. To maintain compatibility with the existing build infrastructure, we created a symbolic link to map the OpenCCA configuration (`defconfig`) to the expected `rockchip_defconfig`, ensuring that the build scripts correctly identify the target parameters.

3.1.3 Kernel Configuration and Instrumentation

The kernel build configuration must be adjusted to support dynamic analysis: observing function execution, resolving pointers, and reading kernel data structures during live AV1 decoding. The default BSP build system overlays configuration fragments from `/bsp/linux/rk2410/` that overwrite manual changes. To ensure custom parameters persisted, we injected them into the `./0001-common/` configuration scripts, guaranteeing their priority during the build process.

Dynamic analysis of the Hantro driver requires three configuration groups: `CONFIG_KALLSYMS_ALL` extends the kernel symbol table to include all internal and unexported functions, making the full driver call graph accessible to `kprobes`, with the exception of inlined functions (Section 2.7.2). `CONFIG_FTRACE` and `CONFIG_KPROBE_EVENTS` enable the dynamic instrumentation infrastructure required for `kprobe` attachment (Section 2.7.2). `CONFIG_BPF` and `CONFIG_DEBUG_INFO_BT` enable the BPF runtime and embed struct type metadata directly in the kernel binary, allowing `bpfftrace` to resolve struct field names at runtime without manual offset calculation (Section 2.7.3). The complete flag listing used in this thesis is provided in Appendix B.1.

3.1.4 Bootloader Configuration and Serial Console Debugging

Deploying the compiled images to the Rock 5B via `scp` over a local network exposed a "silent boot" phenomenon during initial execution. Although the system appeared to hang after U-Boot execution, network analysis confirmed that the kernel had initialized successfully and remained accessible via SSH. This lack of serial output stemmed from a driver mismatch in the `append` line of the boot configuration; the default vendor parameters utilize `console=ttyFIQ0,1500000n8`, which relies on the Rockchip-specific Fast Interrupt Request (FIQ) debugger [Roc24]. While the vendor's downstream 6.1 kernel natively supports this driver, the OpenCCA 6.12 build lacks the requisite FIQ logic alignment and kernel hooks. Therefore, we changed the line to `console=ttyS2`.

3.1.5 GStreamer Build and Configuration

GStreamer is an open-source multimedia pipeline framework that connects modular processing elements, such as decoders, filters, and sinks, into a configurable processing graph [GSt26b]. Integrating GStreamer 1.24.11 provides native support for the `v4l2slav1dec` element [GSt26a], which leverages the V4L2 Request API (Section 2.4.1) to drive hardware-accelerated AV1 decoding. The default system package repository only supplied version 1.22.9; consequently, unlocking this component required us to perform a custom compilation from source. GStreamer manages the pipeline in user space while the Hantro hardware performs the actual decoding;

therefore, every control step crosses the user-kernel boundary as an `ioctl` call, making that boundary the natural point of instrumentation for both the manual analysis (Section 3.2) and dynamic verification (Section 3.5) phases.

To ensure strict version consistency, our implementation compiled the four core GStreamer packages `core`, `plugins-base`, `good`, and `bad` from source via Meson. Disabling automatic features (`-Dauto_features=disabled`) minimized target dependencies.¹ Executing the test pipeline (Listing 4.1) against a 1080p AV1 reference stream verified the successful installation of the V4L2 stateless AV1 video decoder and established the baseline workload for subsequent dynamic analysis.

3.2 Manual Analysis

Establishing a verified ground truth for the driver’s execution flow required us to manually trace the `VIDIOC_REQBUFS` `ioctl` [The24h]. We ran `strace` - to track system calls (Section 2.7.2) - on a GStreamer AV1 decode session and filtered the output, identifying the active set of V4L2 commands. The `VIDIOC_REQBUFS` command served as the primary trace target; it exercises the buffer management subsystem and reaches the driver without triggering the complex interleaving of the full decode pipeline, providing a well-scoped entry point (Figure 4.6).

To cleanly scope the results, our analysis constrained tracing strictly to the media subsystem. Specifically, the V4L2 framework, the VB2 framework, and the Hantro driver (Section 2.3.4). While the manual trace recorded boundary crossings into external domains, such as the memory management and DMA subsystems, it intentionally excluded them from recursive traversal. Section 4.1 provides the complete list of included source files.

Navigating the extensive kernel source tree required a custom NeoVim [Neo24] environment paired with the `clangd` language server [LLV24]. This configuration, documented in Appendix B.2, guaranteed precise jump-to-definition and cross-reference resolution across the kernel source files.

Starting from the `ioctl` `VIDIOC_REQBUFS` entry point, we followed each function call to its implementation and then repeated this process for every function encountered until we reached subsystem boundaries. While the LSP-enabled environment facilitated navigation of direct function calls and structure definitions, the kernel’s `ops` struct pattern (Section 2.5) presented a significant challenge: LSP cannot resolve indirect calls such as `vb2_queue->ops->buf_prepare`, since the concrete implementation is only known at runtime.

¹The kernel’s Contiguous Memory Allocator (CMA) [The24c] poses a potential bottleneck because the VPU DMA engine requires large contiguous regions for frame buffers. While the default configuration proved stable for 1080p decoding, 4K streams or heavily fragmented systems may require manual tuning via `cma=256M`.

We resolved these indirect branches using a hybrid approach that combined manual inspection with static analysis. First, manual inspection of the driver source files located `ops` struct instantiations, such as the `vb2_ops` struct, to map callbacks to concrete functions. Second, a custom CodeQL query systematically discovered all implementations matching specific function pointer types (Section 3.3.4). These workflows operated synergistically: initial manual findings guided the CodeQL query design, and the query results accelerated subsequent source code navigation.

Section 4.2 details the resulting `VIDIOC_REQBUFS` call graph, visualized using diagrams. While this approach established a verified ground truth for an initial `ioctl`, manually scaling it to the remaining 14 target `ioctls` (Section 4.1) was labor-prohibitive. Section 3.3 describes how we leveraged CodeQL to automate this discovery across the full scope of the driver.

3.3 Static Analysis with CodeQL

To scale the manual tracing approach across the full set of target `ioctls`, CodeQL queries are used to perform the static analysis [Git24]. As introduced in Section 2.6, CodeQL treats source code as a relational database for data-flow analysis. We chose it because its declarative language cleanly queries complex call graphs (such as resolving indirect function pointers) without custom traversal code, and predicates easily combine into a reusable library (`hantro_lib`), yielding structured, aggregatable results. The resulting call graph is the foundation for the remainder of the methodology: the struct access analysis in Section 3.4 and the dynamic verification in Section 3.5 both depend on it to scope their search to functions already known to be reachable.

The static analysis methodology over-approximates the execution paths by including all theoretically reachable functions, regardless of whether they execute in practice [NNH99]. This is intentional: it ensures completeness by avoiding false negatives (missing functions that do execute) at the cost of including false positives (functions that never execute). Section 3.5 describes the dynamic verification approach used to distinguish active execution paths from this static superset.

3.3.1 Database Generation and Scoping

CodeQL analysis depends on a relational database generated by intercepting the build process: the CodeQL CLI observes the exact compiler flags, include paths, and macro definitions used for each source file during compilation. We generated the database on an `x86_64` host machine using the ARM64 cross-compilation toolchain, as the process is too resource-intensive to run on the Rock 5B itself.

```
codeql database create hantro-db \
  --language=cpp \
  --overwrite \
  --source-root=bsp/.src/linux \
  --command="make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc) \
  drivers/media/ \
```

Listing 3.2: CodeQL database creation with scoped kernel build

The compilation scoped the build to the full media driver directory (Listing 3.3.1), capturing the architecture-specific macros and conditional compilation blocks relevant to the RK3588 while keeping the database to a manageable size. The initial database scope remains deliberately broad because the CodeQL queries themselves enforce precise subsystem boundaries using the file-path predicates in `hantro_lib` (Section 3.3.2). This eliminates the risk of silently omitting a critical subsystem during the database generation.

3.3.2 Developing the Hantro QL Library

To modularize the analysis and simplify queries, we develop a QL library (`hantro_lib.ql`). This library encapsulates common logic into predicates, accessible using the `Hantro::` namespace. Basic predicates include:

- `isDriverFile(File f)`: Validates if a file is located within the Hantro (`verisilicon`) directory. While a strictly robust approach would identify compilation targets by parsing the `Makefile`, relying on the directory structure is sufficient in this context, as the contents closely mirror the required driver files.
- `isSubsystemFile(File f)`: Checks if a file belongs to the core media subsystems (V4L2, VB2, or the driver).
- `isV4l2Ioctl(Function f)`: Identifies if a function matches the specific `ioctls` identified during the dynamic tracing phase (Section 4.1).

3.3.3 Direct and Indirect Call Invocations

Constructing a recursive call graph from an initial `ioctl` entry point via the `isV4l2Ioctl` predicate requires tracking multiple execution flows. Mapping the Linux kernel's execution path dictates addressing two primary types of invocations:

Direct Calls Standard function calls resolve using CodeQL’s `Call` class. This establishes a direct relationship where the enclosing scope of an invocation matches the caller function, mapping it directly to its statically known target (Listing 3.3, Case 1).

```

predicate calls(Function caller, Function callee) {
  // Case 1: Direct function calls
  exists(FunctionCall site |
    site.getEnclosingFunction() = caller and
    site.getTarget() = callee
  )
  or
  // Case 2: Indirect calls via ops struct function pointers
  exists(ExprCall indirect, Field f |
    indirect.getEnclosingFunction() = caller and
    resolvesFunctionPointerCall(indirect, f, callee)
  )
  or
  // Case 3: Function pointer passed as argument
  // e.g., hantro_return_bufs(q, v4l2_m2m_dst_buf_remove)
  exists(FunctionCall site, FunctionAccess fa |
    site.getEnclosingFunction() = caller and
    fa = site.getArgument(_) and
    callee = fa.getTarget()
  )
}

```

Listing 3.3: CodeQL predicate cases for resolving function calls

Indirect Calls The kernel achieves polymorphism (Section 2.5) through structure-based function pointers, such as `struct vb2_ops`. Resolving an invocation like `vb2_queue->ops->buf_prepare` evades standard static resolution because the specific implementation assignment occurs dynamically. However, CodeQL represents global designated initializers as `ClassAggregateLiteral` objects (Section 2.6.1). A custom two-step static resolution bridge connects an indirect call site to a concrete implementation using two complementary predicates:

- **resolvesOpsField(Field f, Function callee)** defines the *instantiation boundary*. This predicate scans for aggregate initializers, as the driver framework registers callback implementations using static global structs (such as `static const struct vb2_ops hantro_qops`). It matches these structs to map the target field `f` (specifically, `queue_setup`) directly to the concrete `callee` function (specifically, `hantro_queue_setup`) assigned within the subsystem files.
- **resolvesFunctionPointerCall(ExprCall c, Field f, Function callee)** identifies the *invocation boundary*. Given an indirect control flow instruction such as `q->ops->queue_setup(...)`, this predicate isolates the pointer field access to extract the target field `f`. It then leverages **resolvesOpsField** to resolve that field to its mapped `callee` (Listing 3.3, Case 2).

Furthermore, the kernel frequently passes function pointers as arguments to higher-order functions such as `hantro_return_bufs(q, v4l2_m2m_dst_buf_remove)`, where the callee later invokes the passed callback. To capture this pattern, the static analysis model introduces a third resolution case identifying when a function is passed by reference (Listing 3.3, Case 3).

Combining these direct and indirect logic paths yields a unified `calls(caller, callee)` predicate. Applying transitive closure by adding `*` before the predicate call maps the entire downstream execution tree (Section 2.6.1). The full query is provided in Appendix C.1. Validation against the manual trace (Section 4.3) initially revealed missing functions, prompting the discovery and integration of Case 3. Following this addition, the static analysis model achieved full coverage of the manual ground truth, confirming the sufficiency of these three resolution strategies.

3.3.4 Filtering Indirect Calls

The unfiltered call graph introduced in Section 3.3.3 over-approximates the execution paths by including implementations from every driver in the database scope. For example, querying all implementations of `struct vb2_ops` returns callbacks from USB video, software codecs, and other unrelated drivers alongside the AV1-specific ones. To isolate the active execution path, we introduce a filter predicate `isImplementedFunction` into the `calls` predicate: an indirect call is only followed if it resolves to a function we have positively identified as part of the Hantro AV1 decoding pipeline.

Populating this filter requires systematically discovering every `ops` struct the driver uses, which we find iteratively using a dedicated "Discovery" query. The workflow proceeds as follows:

1. Run the Discovery query to traverse the unfiltered call graph. It uses the `callsUnfiltered*` predicate (which omits the `isImplementedFunction` filter because that data is still being populated) to find all indirect calls resolving to an `ops` struct that has not yet been captured.
2. For each newly discovered `ops` struct, find its possible implementations using the `resolvesOpsField` predicate and determine the correct Hantro-specific implementation by reviewing the source code or driver documentation.
3. Add all functions that the implementation registers to `isImplementedFunction`, and add the `ops` struct type to `isTrackedOpsStruct`.
4. Rerun the Discovery query. Because `isTrackedOpsStruct` now covers this structure, it no longer appears in the results.
5. Repeat until the Discovery query returns zero results.

When the query returns zero results, every `ops` struct reachable from the target `ioctl`s has been accounted for, and the filtered `calls` predicate no longer follows any indirect call to an unverified implementation. This means we have constrained the set of `ops` structs to be on the active path, increasing the precision. Section 4.3 discusses the completeness of this process by cross-validating against the manual ground truth. This confirmed that no reachable `ops` structs were missed. The full implementation of the Discovery query is provided in Appendix C.2.

We determined the correct Hantro implementation for each `ops` struct by examining naming conventions, file location, and driver documentation. Section 3.5 therefore confirms these implementation assumptions at runtime. Additionally, it identifies which statically reachable functions are never actually invoked during a real decode workload. To scale this confirmation across all discovered functions, Section 3.5.2 describes a Python script that parses the Discovery query output and generates `bpfttrace` probe scripts for all traceable implementations. This automated dynamic analysis can be used directly instead of step 2 above.

3.4 Shared Struct Access Analysis

While the call graph analysis (Section 3.3) identifies which functions execute across the kernel-driver boundary, understanding the driver’s full kernel interleaving requires mapping its data dependencies. A subsequent query identifies the specific kernel structs that the Hantro driver reads or writes during execution. This defines the structural scope required to isolate the driver while retaining its functionality.

To capture this scope, this thesis employs two parallel analysis strategies: syntactic field access and global taint tracking (Section 2.6.3). Theoretically, these methods are strictly complementary. The syntactic approach establishes a conservative structural upper bound by capturing all potentially accessible elements, regardless of whether the path executes at runtime. Conversely, taint tracking isolates only those elements with verified data flow across the user-kernel boundary.

Cross-referencing, these methods should ideally yield a perfect structural map; however, this theoretical synergy does not hold in practice. As evaluated in Chapter 4, static analysis limitations cause the taint tracking model to both omit critical target structures due to flow breaks and inadvertently incorporate structures from entirely unrelated codecs.

3.4.1 Syntactic Field Access

The syntactic query identifies every struct type accessed across the kernel–driver boundary within the `ioctl-reachable` call graph. It uses the file-path predicates from `hantro_lib` (Section 3.3.2) to assign each function and struct to either the driver or kernel domain, then restricts the search to functions reachable via the filtered call graph (Section 3.3.3), thus already filtering structs from other codecs.

For each reachable function, the query enumerates all `FieldAccess` nodes in its body and checks whether the accessed struct belongs to a different domain than the accessing function. If so, it records the struct name and classifies the access using CodeQL’s `isModified()` predicate: accesses that alter the struct are flagged as `Modified`; all others as `Read-only`. A `direction` string encodes which domain crosses into which (`Driver -> Kernel` or `Kernel -> Driver`) (Listing C.3).

This method captures every structural reference in the reachable call graph, regardless of runtime reachability, producing a complete set that includes dead code paths, such as inactive error recovery handlers.

3.4.2 Dataflow Taint Tracking

Taint tracking traces how specific values (such as kernel structure pointers) flow from a defined source to a defined sink. It only reports structures that a pointer actually reaches during `ioctl` execution. Taint tracking aims to uncover struct accesses invisible to syntax alone, such as unresolved opaque casts. We model three crossing patterns based on the V4L2/VB2 architecture described in Section 2.3.4:

- **Kernel $\rightarrow$ Driver (K $\rightarrow$ D):** The kernel invokes a driver-registered callback (such as a `vb2_ops` function) and passes a kernel-owned struct pointer as an argument. The query tracks field accesses on that parameter within the driver.
- **Driver Query $\rightarrow$ Kernel (DQ $\rightarrow$ K):** The driver retrieves a kernel pointer via accessors such as `hantro_get_ctrl` to read per-frame decode parameters from the V4L2 Request API (Section 2.4.1). The query tracks field accesses on the returned pointer within the driver.
- **Driver $\rightarrow$ Kernel (D $\rightarrow$ K):** The driver passes a pointer to a driver-owned structure into a kernel function. The query tracks field accesses on that pointer within the kernel.

Standard CodeQL taint tracking does not propagate flow through several C patterns common in the kernel: struct field access through a pointer, `ops` struct dispatch, the `container_of` macro, and union field access for codec-specific control types.

Without modeling these explicitly, flow breaks prematurely, and structures go undetected. We therefore register four custom flow steps via a shared `sharedFlowStep` predicate to handle each case. The analysis is bound to the `ioctl1-reachable` call graph as in Section 3.3.

3.5 Dynamic Verification using *bpftrace*

Since static analysis provides complete coverage of all theoretically reachable execution paths, it includes functions that never execute during AV1 decoding workloads. To validate which paths are active, we instrument the running kernel using *bpftrace*, which is a tracing language that enables dynamic instrumentation of kernel functions at runtime (Section 2.7). This serves two purposes: confirming which `ops` struct implementations the AV1 decoder actually uses and identifying which statically reachable functions are invoked during decoding. This eliminates false positives from the over-approximated call graph.

As detailed in Section 2.7.3, executing this dynamic verification relies on two kernel configurations: BTF type information to dereference struct fields by name at runtime, and `CONFIG_KALLSYMS_ALL=y` to expose all kernel symbols to `kprobe` attachment. The results of this runtime validation are presented in Section 4.6.

3.5.1 Handling Inlined Functions

Standard *bpftrace* probes (such as *bpftrace -e 'kprobe:f @ = count(); '*) attach to function entry points by symbol name. However, functions that the compiler inlines have no such entry point and are therefore invisible to `kprobes` (Section 2.7.2).

Tracing inlined functions requires attaching probes at call sites rather than function entry points. While tools such as `perfprobe` can resolve inlined functions automatically [Gre14], implementing this reliably across all 159 untraceable functions falls outside the scope of this thesis. The analytical impact is limited in any case, as most inlined functions are small helpers whose execution status does not affect the identification of unused code paths.

Resolving `ops` struct implementations. For `ops` struct callbacks specifically, rather than probing the callback itself, we intercept an attachable caller that holds a reference to the struct, read the relevant function pointer field by name using BTF, and resolve it to a symbol via `ksym()` (Listing 3.4). This directly reveals which concrete implementation the driver registered without requiring the callback to be traceable. The goal is not execution counting but validating the implementation assumptions made during iterative filtering (Section 3.3.4).

```
#include <linux/media/videobuf2-core.h>

kprobe:target_kernel_function
{
    /* With BTF and CONFIG_KALLSYMS_ALL, we can resolve implementation pointers */
    $struct_ptr = (struct vb2_queue *)arg0;
    $func_ptr = $struct_ptr->ops->buf_queue;

    printf("Implementation resolved to: %s\n", ksym($func_ptr));
}
```

Listing 3.4: bpftrace example for resolving function pointers via BTF

3.5.2 Automated Trace Generation

To scale verification across all functions identified by the static analysis, we developed a Python script that parses the Discovery query output (Section 3.3.4) and automatically generates **bpftrace** probe scripts.

For each function in the query output, the script cross-references its name against `/sys/kernel/debug/tracing/available_filter_functions` on the Rock 5B, which lists all functions currently available for **kprobe** attachment; functions absent from this file are inlined or unexported and are appended as comments in the generated script.

For traceable functions, the script emits an invocation-counting **kprobe**, as shown in Listing 3.5.

Executing the probes during an active AV1 decode session (Listing 4.1) captured the invoked functions, which subsequently populated and validated the filter implemented in Section 3.3.4. Instead of limiting the pipeline to the initial Discovery query, extending the generator to use the full subsystem call graph scaled the analysis across the entire static result set. This broader scope provided deeper insight into which execution paths remain dormant during a standard decoding workload, enhancing overall analysis precision. We skipped untraceable functions within this extended scope (Section 3.5.1), with a more comprehensive resolution deferred to future work (Section 5.4).

```
kprobe:hantro_buf_queue
{
    @vb2_ops["hantro_buf_queue calls"] = count();
}
```

Listing 3.5: Example of a generated kprobe for invocation counting

4 Results

This chapter presents the results of the three-stage analysis methodology described in Chapter 3: Section 4.1 defines the analysis scope and lists the set of target `ioctls`. Section 4.2 presents the call graphs for the `REQBUF` and all target `ioctls`. Section 4.3 validates the methodology by comparing the manually traced ground truth against static analysis results. Section 4.4 finds all subsystems that are called by, or are calling the driver. Section 4.5 shows the implemented operations structs (`ops` structs) for AV1 decoding. Section 4.6 analyzes dynamic execution results, distinguishing active code paths from unreachable over-approximations. Finally, Section 4.7 identifies the shared structs used by the driver and kernel.

4.1 Analysis Scope and Target `ioctls`

The analysis focused on the media subsystems directly involved in AV1 hardware decoding. We deliberately constrained the scope to the following source directories:

- **V4L2 Core:** `drivers/media/v4l2-core/` (`ioctl` dispatch, control framework, M2M framework, device registration, Section 2.3)
- **Videobuf2 Framework:** `drivers/media/common/videobuf2/` (buffer life-cycle management, memory backends, Section 2.3.2)
- **Hantro Driver:** `drivers/media/platform/verisilicon/` (hardware-specific decode logic, DMA programming, Section 2.4.2)

When a function call crossed outside these directories, such as into memory management (`alloc_pages`) or interrupt handling (`request_irq`), we recorded it but did not trace it further. These crossings represent dependencies on generic kernel infrastructure rather than media-subsystem-specific logic, and tracing into them would expand the scope beyond what is relevant for driver extraction.

Target `ioctls` Tracing a complete GStreamer AV1 decode session (Section 3.1.5) with `strace` established the target `ioctl` set. This Linux diagnostic tool intercepts and logs all system calls made by a process, recording the call name, arguments, and return value for each invocation (Section 2.7.2).

```
gst-launch-1.0 filesrc location=~/Sintel_1080_10s_10MB.mp4 ! \
qtdemux ! aviparse ! v4l2slav1dec ! fakesink
```

Listing 4.1: Pipeline to execute hardware-accelerated AV1 decode

We ran the pipeline against the 10-second 1080p input file, derived from the Sintel short film¹, over a session. Table 4.2 lists all 15 `ioctl`s observed, together with their call counts and whether the `ioctl` has a driver-side implementation in the Hantro verisilicon driver. The driver column in Table 4.2 categorizes the trace’s `ioctl`s by their execution paths. First, the eleven `ioctl`s registered in the `v4l2_ioctl_ops` struct serve as the driver-implemented entry points for our static analysis (Section 3.3). Second, the kernel handles media request and synchronization `ioctl`s entirely outside the V4L2 framework, contributing nothing to the driver call graph. Third, the driver leaves unimplemented `ioctl`s `NULL` (Section 4.5), causing the V4L2 core to safely return `-EINVAL` without executing any driver code or generating an analysis path.

Table 4.2: `ioctl`s observed during the GStreamer AV1 decode run.

<code>ioctl</code>	Purpose	Calls	Driver
<code>VIDIOC_QUERYCAP</code>	Capability query	4	✓
<code>VIDIOC_ENUM_FMT</code>	Format enumeration	22	✓
<code>VIDIOC_G_FMT</code>	Get format	7	✓
<code>VIDIOC_S_FMT</code>	Set format	6	✓
<code>VIDIOC_REQBUFS</code>	Request buffers	4	✓
<code>VIDIOC_QUERYBUF</code>	Query buffer status	14	✓
<code>VIDIOC_QBUF</code>	Queue buffer for decode	614	✓
<code>VIDIOC_DQBUF</code>	Dequeue completed buffer	614	✓
<code>VIDIOC_STREAMON</code>	Start streaming	2	✓
<code>VIDIOC_STREAMOFF</code>	Stop streaming	2	✓
<code>VIDIOC_S_EXT_CTRL</code>	Set extended controls	308	✓
<code>MEDIA_REQUEST_IOC_QUEUE</code>	Submit decode request	307	
<code>MEDIA_REQUEST_IOC_REINIT</code>	Reinitialise request object	307	
<code>DMA_BUF_IOCTL_SYNC</code>	DMA-BUF cache synch.	1814	
<code>VIDIOC_QUERYCTRL</code>	Query control	0	

The call counts reflect the stateless decode model (Section 2.4): each decoded frame requires one `OUTPUT` buffer submission and one `CAPTURE` buffer retrieval, producing paired queue and dequeue calls. Each frame also requires a media request submission carrying the per-frame AV1 control parameters, with matching reinitialization calls confirming that request objects are recycled after each submission. The elevated synchronization call count arises from bracketing each CPU

¹https://test-videos.co.uk/vids/sintel/mp4/av1/1080/Sintel_1080_10s_10MB.mp4

access to shared DMA buffers, as required by the DMA-BUF sharing model (Section 2.3.2).

4.2 Visualization of the Call Graphs

This section derives the presented call graphs from the manual trace of `VIDIOC_REQBUFS` and the CodeQL static analysis. The manual trace results are first presented at two levels of granularity and then compared against the full static trace to illustrate the increased complexity across all target `ioctl`s.

Figure 4.3 collapses the call graph to the level of source files, grouping less central files into composite nodes: peripheral V4L2 helpers are combined into *Other V4L2*, and all Hantro driver files except the AV1-specific decode logic are grouped into *Hantro Driver*, with the AV1 decode files represented separately as *AV1 Decoder*. Edges represent at least one call between functions in the connected files. This abstraction visualizes the subsystem structure: the dominant interaction is between the Hantro driver and the VB2 core, while the remaining files act as one-way callees with no callbacks into their callers. The absence of back-edges from these peripheral files confirms that `VIDIOC_REQBUFS` follows a straightforward dispatch chain: the V4L2 core validates and routes the call, VB2 manages the buffer allocation lifecycle, and the driver is consulted only to provide hardware constraints such as minimum buffer counts and plane size requirements.

Figure 4.4 shows the complete function-level call tree for the same trace. Nodes are grouped into clusters by source file, making the subsystem decomposition from Figure 4.3 visible at full resolution.

While the isolated trace of `VIDIOC_REQBUFS` yields a predictable linear dispatch chain, overlaying the full framework coverage reveals a far more interconnected call graph. Figure 4.5 expands the composite file groupings of Figure 4.3 into their individual constituent files, overlaying the complete static coverage (dashed lines) onto the baseline `VIDIOC_REQBUFS` flow (solid lines). This finer-grained view highlights three architectural observations:

- **Shallow Driver Penetration during Allocation:** During a `VIDIOC_REQBUFS` call, execution only reaches the driver’s outermost interface (`hantro_v4l2.c`), which queries hardware constraints and returns immediately to the VB2 core. The driver core (`hantro_drv.c`) is not touched during buffer allocation.
- **Deep AV1-Specific Logic during Decode:** The full coverage map reveals that active decoding reaches into AV1-specific codec files. This includes film grain synthesis helpers, which regenerate noise from parameters signaled in the bitstream (Section 2.4.3). This represents hardware-specific decode logic that `VIDIOC_REQBUFS` never approaches.

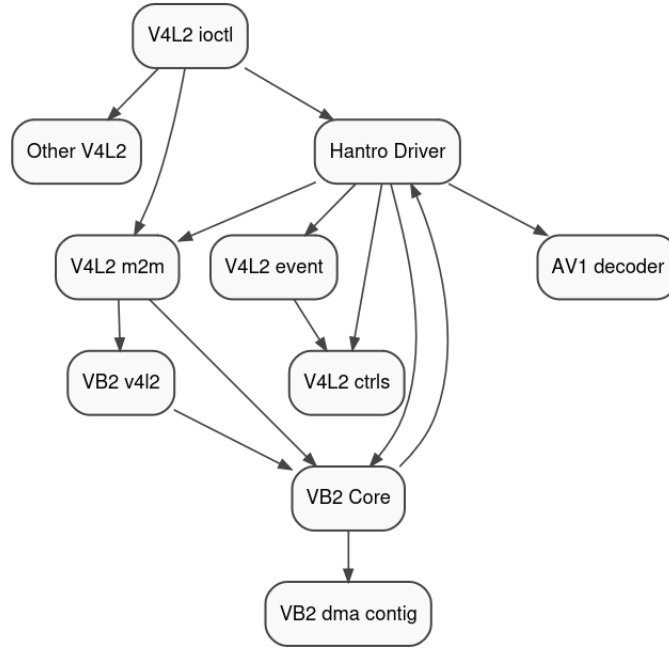
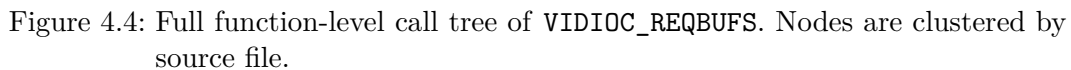


Figure 4.3: File-level call graph of `VIDIOC_REQBUFS`. Edges represent one or more calls between functions in the connected files.

- **Bidirectional Coupling in the Streaming Path:** The full topology exposes `hantero_drv.c` and the post-processor, which is the optional hardware stage that converts decoded output into a display-ready pixel format (Section 2.4.3), as heavily coupled with the V4L2 M2M, control, and event subsystems. Unlike the strict downward dispatch of `VIDIOC_REQBUFS`, the streaming phase triggers callbacks into these subsystems to manage state transitions and hardware interrupts.

Figure 4.6 shows the number of distinct reachable functions per `ioctl`, broken down by analysis scope. Two `ioctls` stand out as disproportionately complex: `v4l_qbuf` and `v4l_streamon` together account for the largest function counts, confirming that the buffer submission and stream lifecycle management dominate the analysis. The large subsystem counts for both `ioctls` reflect deep VB2 and V4L2 core involvement, while the Kernel column captures shared infrastructure outside the media subsystem, such as locking and memory management primitives. By contrast, `v4l_reqbufs` has a driver footprint of only 13 functions, which is precisely why we selected it as the ground truth baseline: it is tractable for exhaustive manual verification while still exercising all four subsystem boundary types. The zero Driver counts for `v4l_`



4.3 Methodology Validation

To validate the static analysis, we compare it against the manually traced ground truth for `VIDIOC_REQBUFS` (approximately 100 functions, Section 3.2) and the CodeQL call graph for the same `ioctl` (approximately 180 functions). The difference in raw counts is expected: static analysis resolves macros and aliased symbols that

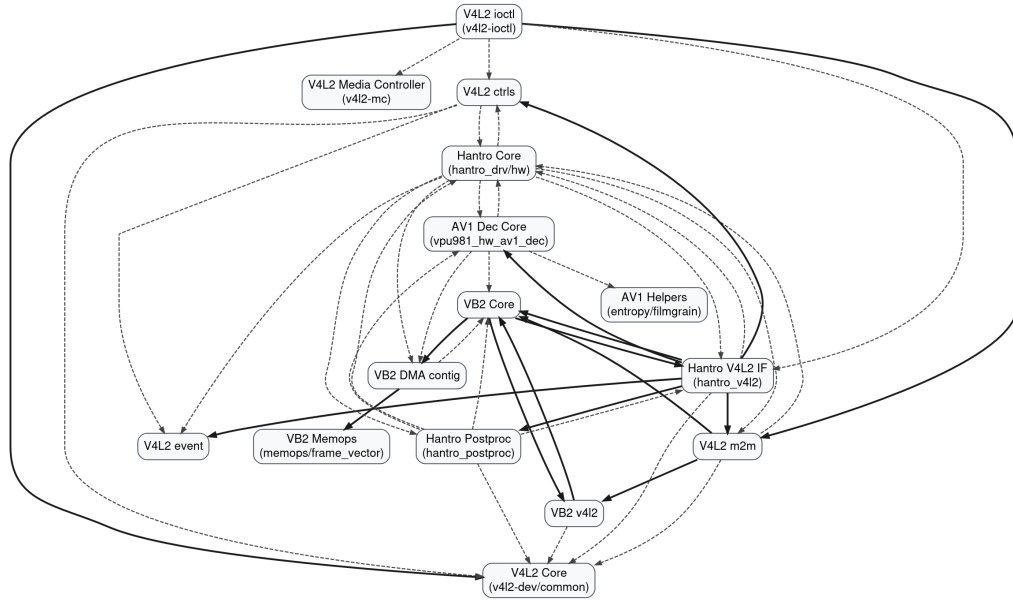


Figure 4.5: Overlay of the full driver-framework topology (dashed) against the isolated `VIDIOC_REQBUFS` execution path (solid). Each node represents an individual source file.

a human naturally abstracts away. The comparison therefore targets logical discrepancies (functions absent from one for a non-trivial reason), rather than exact numerical agreement.

Table 4.7: Discrepancy categories between manual and static traces.

Category	Origin	Resolution
Conditional check misinterpretation	Manual error	Filter
Function pointer argument pattern	Query gap	Add Case 3
Typographical errors	Manual error	Correct
Preprocessor macros	Abstraction gap	Filter
Subsystem aliasing	Abstraction gap	Note
Inactive hardware branches	Over-approximation	Iterative filtering

The comparison reveals six discrepancy categories, summarized in Table 4.7 and detailed in full, with representative functions, in Appendix D. Two originate from human error in the manual trace: a misinterpreted conditional and a typographical mistake. One is a genuine gap in the CodeQL query (the missing function-pointer-argument pattern), which we close by adding Case 3 to the `calls` predicate (Section 3.3.3). One reflects the over-approximation of static analysis, which includes

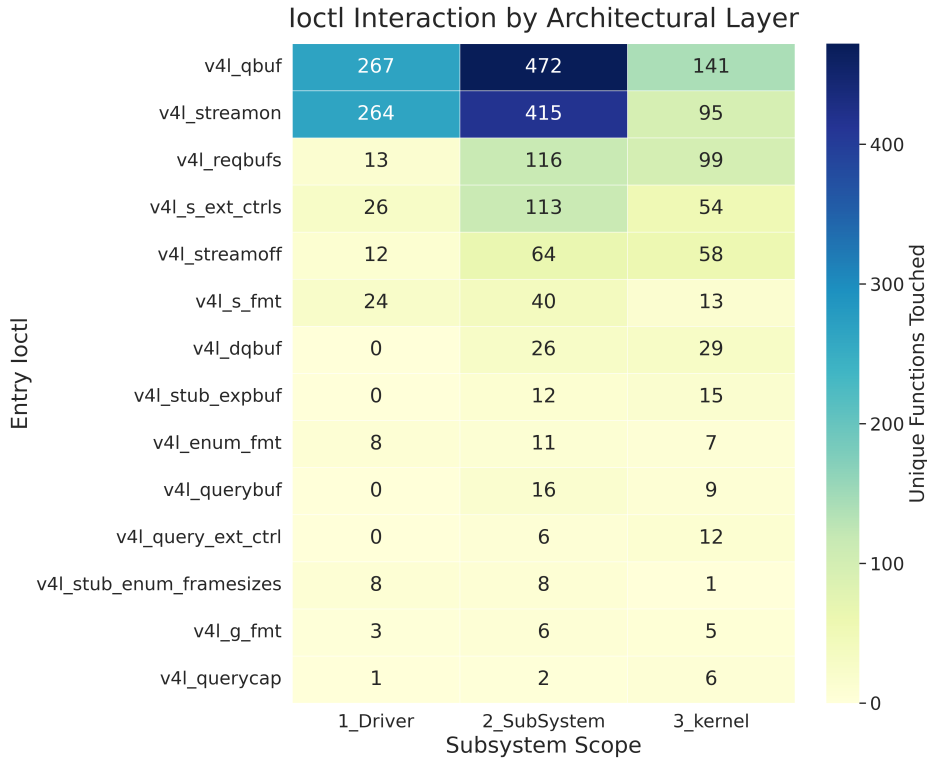


Figure 4.6: Number of distinct reachable functions per ioctl, broken down by scope: Driver (`verisilicon/`), subsystem (`V4L2` and `VB2` directories), and Kernel (all other kernel code outside the defined analysis scope).

inactive hardware branches that are never taken at runtime. The remaining two are abstraction-level mismatches: preprocessor macros and subsystem aliasing. These arise from comparing a human-readable trace to a compiled one, rather than from an error on either side. Of these six categories, the query gap is the one finding with a methodological consequence during the validation: it was discoverable only because the manual trace served as an independent reference, and it directly motivated Case 3 of the `calls` predicate. The remaining categories are reconciliation overhead rather than analysis flaws; their broader implications for the reliability of manual tracing are discussed in Section 5.3.

4.4 Kernel Dependencies for AV1 Decoding

The call tree query processed all `ioctl` entries (see Section 3.3.3). To simplify the findings, the analysis tracked calls exclusively at the file level and filtered them to isolate interactions between driver and non-driver files.

Table 4.8 summarizes these architectural dependencies, grouping interactions by kernel subsystem. Within this summary, *Dir.* denotes the communication direction between the Driver (D), Kernel (K), and Subsystem (S), marked as unidirectional ($\rightarrow$) or bidirectional ($\leftrightarrow$). *Max reach* indicates the highest number of distinct `ioctl` paths, while *Union* represents the unique `ioctls` across all edges for that subsystem group. For the exhaustive, unaggregated module-to-module call mapping, see Table E.1 in the appendix. The identified cross-boundary edges fall into three structural categories, mapping directly to the core kernel and media infrastructures detailed in Sections 2.2–2.4.

Direct Kernel Dependencies

These connections represent core execution and allocation services invoked outside of the media-framework boundaries and can be compared to Section 2.2.

DMA Subsystem (`dma-mapping.h`) Allocates backing memory for the output surface during buffer setup via `dma_alloc_attrs` and flushes hardware caches on every decode execution path (QBUF/STREAMON). The traces confirm its involvement in enforcing cache coherency over physical memory layers.

Diagnostics and Utilities (`printk.h`, `string_fortified.h`) Represents the largest dependency by total `ioctl` path volume. The query records continuous execution across the driver lifecycle for diagnostic logging and memory block operations via `memcpy` and `memset`.

Runtime PM and Clock Control (`pm_runtime.h`, `clk.h`) Appears strictly on the job-dispatch path during active pipeline states (QBUF/STREAMON) to gate the underlying hardware clock trees during code execution.

Workqueue and Timing (`workqueue.h`, `jiffies.h`) Triggers `queue_work` upon job completion to defer interrupt bottom-half processing, while simultaneously evaluating hardware execution timeouts using the kernel `jiffies` counter.

Slab Allocator (`slab.h`) Handles dynamic memory configuration calls during active decoding cycles (QBUF/STREAMON) to instantiate per-frame AV1 tile maps and segment structures.

Memory-Mapped I/O (`io.h`) Executes direct hardware register access via `readl` and `writel` abstractions at the driver boundary. This interaction occurs solely during job submission to update hardware configuration registers.

Media Framework Dependencies

These interactions mark structural framework bindings, capturing outward utility requests alongside expected subsystem callback sequences (Section 2.3.4).

- VB2 Core (videobuf2-core.h)** A bidirectional boundary managing the multi-stage queue and frame lifecycle outlined in Section 2.3.2. The driver checks internal VB2 buffer tracking states, while the framework triggers driver-specific execution operations through the `vb2_ops` callback layout.
- V4L2 Mem2Mem (v4l2-mem2mem.c/.h)** Establishes the bidirectional boundary that coordinates frame execution (see Section 2.3.3). The framework coordinates hardware concurrency via the driver's `device_run` routine, while the driver calls out to `v4l2_m2m_job_finish` to return processed frames.
- V4L2 Core Dev and Helpers (v4l2-dev.c/.h)** Consists entirely of outbound calls from the driver to access stateless utility functions. The trace maps these operations to capability verifications (`QUERYCAP`) and pixel format confirmation (`S_FMT`), running without modifying driver state variables.
- V4L2 Controls (v4l2-ctrls-core.c)** Provides the bidirectional data channel for ingesting stateless, per-frame AV1 metadata arrays (see Section 2.4). The interface targets the `v4l2_ctrl_ops.s_ctrl` callback path to supply config structures to the driver context.
- V4L2 Event (v4l2-event.c)** Handles internal queues for frame status and source modification notifications. The dynamic analysis indicates that this interface maintains zero calls during active decoding operations (Section 4.6).
- VB2 DMA-Contig Allocator (videobuf2-dma-contig.h)** Maintains outbound edge references to fetch memory layout profiles and execute inline configuration definitions necessary for contiguous buffer tracking.

Framework Callbacks into the Driver

- V4L2 ioctl Dispatch (v4l2-ioctl.c)** Channels inbound execution paths to govern top-level parameter ingestion, directing media operations down to specific driver-backed handlers (see Section 2.3.4). Every evaluated active `ioctl` path passes through this framework boundary, referencing the entry points declared within the `v4l2_ioctl_ops`, `vb2_ops`, and `v4l2_ctrl_ops` structures.

4.5 Hardware-Specific ops Struct Implementations

The Linux media subsystem relies on object-oriented paradigms implemented in C via structures containing function pointers commonly denoted as `ops` structs (Section 2.5). The analysis identified 13 `ops` structs that are relevant to the video decoding pipeline during static analysis.

Table 4.8: Aggregated cross-boundary call edges between the driver and the non-driver files.

Dir.	Module / Subsystem	Driver files involved	Max reach	Ioctl union
D → K	DMA subsystem	hantro_postproc.c, rockchip_vpu981_hw_av1_dec.c	4	4
D → K	Printk / string / error	hantro.h, hantro_v4l2.c, rockchip_av1_entropymode.c, rockchip_vpu981_hw_av1_dec.c	4	8
D → K	Runtime PM / clock	hantro_drv.c	2	2
D → K	Workqueue / timing	hantro_drv.c	2	2
D → K	Slab allocator	rockchip_vpu981_hw_av1_dec.c	2	2
D → K	MMIO	hantro.h	2	2
D ↔ S	VB2 core	hantro_v4l2.c, hantro_postproc.c, rockchip_vpu981_hw_av1_dec.c	4	6
D ↔ S	V4L2 mem2mem	hantro.h, hantro_drv.c, hantro_v4l2.c	4	6
D → S	V4L2 core (dev/common)	hantro_v4l2.c, hantro_postproc.c	4	5
D ↔ S	V4L2 controls	hantro_drv.c, hantro_v4l2.c	2	4
D → S	V4L2 event	hantro_drv.c, hantro_v4l2.c	2	4
D → S	VB2 DMA-contig alloc	hantro.h, rockchip_vpu981_hw_av1_dec.c	2	2
S → D	V4L2 ioctl dispatch	hantro_v4l2.c	1	7

A targeted query mapped these generic `ops` structs to their possible implementations (Section 3.3.4). Cross-referencing the dynamic invocation logs (Appendix F.1) against the static filtering results verified the active implementation of each `ops` struct. For Hantro-specific handlers, such as `hantro_buf_queue` or `rockchip_vpu981_av1_dec_run`, a single dynamic invocation sufficiently validated the static mapping. Conversely, several `ops` structs link to generic VB2 helper functions—such as `vb2_ops_wait_prepare` or `vb2_dc_finish`—which are shared across multiple drivers. In these instances, a single fired function could not unambiguously identify the parent `ops` struct instance. Therefore, validating these generic structures required confirming that multiple fired callbacks originated from the same `ops` struct, ensuring the full set of registered functions matched the expected runtime invocation pattern. Isolating the active implementation for each `ops` struct and filtering out extraneous branches restricts the resulting call graph to the Rockchip RK3588 AV1 decoding context. The mapped implementations are:

- **vb2_ops:** Implemented by `hantro_queue_ops` defined in `hantro_v4l2.c`. It manages the core lifecycle of video buffers.
- **hantro_codec_ops:** The hardware-specific codec interface, implemented specifically for the RK3588 AV1 decoder in `rockchip_vpu981_hw_av1_dec.c`.
- **vb2_mem_ops:** Implemented by the DMA contiguous memory allocator (`vb2_dc_*`) in `videobuf2-dma-contig.c`, as required by the Hantro VPU's hardware DMA constraints (Section 2.3.2).
- **vb2_buf_ops:** Implemented by `v4l2_buf_ops` in `videobuf2-v4l2.c`, which manages V4L2-specific buffer validations and initializations.
- **v4l2_ioctl_ops:** Implemented by `hantro_ioctl_ops` in `hantro_v4l2.c`, which routes user-space `ioctl` commands to the driver.
- **v4l2_ctrl_ops:** Implemented by `hantro_av1_ctrl_ops` in `hantro_drv.c`, handling AV1-specific V4L2 controls.
- **v4l2_ctrl_type_ops:** Implemented in the core `v4l2-ctrls-core.c` framework for type-specific control operations.
- **v4l2_m2m_ops:** Implemented by `vpu_m2m_ops` in `hantro_drv.c`, handling the memory-to-memory job scheduling.
- **hantro_postproc_ops:** Handles optional post-processing features on the Rockchip hardware.
- **media_request_object_ops:** Utilized by two distinct ops structs for unbinding requests (`vb2_req_unbind` and `v4l2_ctrl_request_unbind`).

Furthermore, the dynamic trace monitored three additional ops structs: `v4l2_subscribed_event_ops`, `v4l2_subdev_video_ops`, and `v4l2_subdev_pad_ops`. Whose callbacks registered zero runtime invocations throughout the decode session. This absence confirms that while these structures are statically reachable and registered by the driver, they remain unexercised during standard AV1 decoding and can be excluded from the active call graph.

Many functions defined within these ops structs are optional. Driver maintainers frequently leave these function pointers set to NULL (e.g., `g_volatile_ctrl` in `v4l2_ctrl_ops`, or `job_abort` and `job_ready` in `v4l2_m2m_ops`). As they are absent from the concrete implementation struct, the `isActiveImplementation` predicate finds no assignment for these fields and produces no call edge. The static analysis, therefore, only follows calls to functions explicitly registered in the driver's ops structs.

4.6 Dynamic Validation via bpftrace

Executing the generated `bpftrace` scripts (Section 3.5.2) against the identical 10-second Sintel AV1 decode session used throughout this analysis (Listing 4.1). Of the 367 functions predicted by the static analysis, 159 could not be instrumented because they are absent from `available_filter_functions`. They are mostly inlined functions that the compiler resolved without emitting an addressable symbol, as discussed in Section 3.5.1. We successfully attached probes to the remaining 208 functions; of these, 164 fired at least once, and 44 registered probes but produced zero hits. The complete per-function invocation counts are provided in Appendix H.1.

4.6.1 Confirmed Execution Paths

The invocation counts align directly with the structural properties of the V4L2 M2M subsystem and the 307-frame workload, providing a coherent picture of the driver’s runtime behavior. Dynamic confirmation of the function-pointer-dispatched callbacks within this path is discussed in Section 4.5.

Frame-level alignment. The core decode functions, including `rockchip_vpu981_av1_dec_run` and `device_run`, each fired exactly 307 times, confirming a strict one-to-one correspondence between decoded frames and hardware scheduling jobs. Setup helpers such as `hantro_start_prepare_run` match this count exactly, as does the IRQ completion callback `hantro_irq_done`. The entire V4L2 controls pipeline—comprising request setup, completion, and binding functions such as `v4l2_ctrl_request_setup`—likewise fires 307 times per decode cycle.

Dual-queue symmetry. The M2M framework maintains two independent queues: OUTPUT for compressed input and CAPTURE for decoded output (Section 2.3.3). Lifecycle functions `hantro_queue_setup`, `hantro_start_streaming`, and `hantro_stop_streaming` each fired exactly twice, once per queue. Buffer-level functions `hantro_buf_prepare`, `hantro_buf_queue`, `__buf_prepare`, and `__enqueue_in_driver` fired 614 times (307 frames $\times$ 2 queues), confirming symmetric buffer processing across both queues for every decoded frame.

Buffer pool allocation. The DMA-contiguous allocator `vb2_dc_alloc` and its associated export function `vb2_dc_get_dmabuf`, which exports an MMAP-allocated buffer as a DMA-buf file descriptor for zero-copy display handoff (Section 2.3.2). Each fired exactly 14 times, matching the `VIDIOC_QUERYBUF` count in Table 4.2 and confirming that the application allocated a fixed pool of 14 reusable buffers at initialization rather than allocating per frame. The high call counts of `vb2_dc_cookie`

and `vb2_plane_cookie` (2,679 each) reflect the continuous DMA mapping lookups required beneath the higher-level buffer operations.

Post-processor state. The Hantro post-processor is an optional hardware stage that converts the decoder’s native output into a display-ready pixel format (Section 2.4.3). `rockchip_vpu981_postproc_disable` fired exactly once per decoded frame (307 times), confirming that the driver resets the post-processor at every frame boundary. `rockchip_vpu981_postproc_enable` never fired, confirming that the RK3588 post-processor is unconditionally bypassed for this AV1 decode configuration. This is consistent with the `hantro_needs_postproc` call count of 2,701, indicating that the predicate is evaluated repeatedly but always returns false for this codec and pixel format combination.

4.6.2 Statically Reachable but Not Executed

Forty-four functions had probes registered but fired zero times during the decode session. These fall into four groups, each reflecting a distinct reason why a statically reachable path was not exercised by the test workload.

Post-processor enable path. Six functions forming the post-processor bring-up sequence (including `hantro_postproc_enable` and `rockchip_vpu981_postproc_enable`) remain uncalled. As established above, the post-processor is never activated for AV1 on this configuration; the enable path is therefore statically reachable but dynamically dead for this workload.

Alternative memory backends. VB2 buffers support three allocation modes: MMAP, USERPTR, and DMABUF (Section 2.3.2). The GStreamer `fakesink` pipeline used for validation does not share decoded frames with any other hardware subsystem, meaning DMABUF’s zero-copy inter-subsystem sharing provides no benefit; MMAP is sufficient and simpler to configure. As a result, fifteen functions covering the DMABUF import and USERPTR modes of the VB2 DMA-contig allocator remain inactive. These include the `vb2_dc_map_dmabuf` and `vb2_dc_get_userptr` families, along with VB2 core dispatch entries such as `__prepare_dmabuf`. These represent correct static over-approximations: the `vb2_dc_mem_ops` struct registers all memory-backend callbacks, but only the MMAP callbacks execute in this pipeline configuration.

Film grain synthesis. AV1 film grain synthesis regenerates noise on playback from parameters signaled in the bitstream rather than compressing the grain directly (Section 2.4.3). `rockchip_av1_generate_luma_grain_block` and `rockchip_av1_generate_chroma_grain_block` implement the software-side AV1 film grain synthesis path. Their absence indicates that the test bitstream does not carry a film grain parameter set. Notably, `rockchip_vpu981_av1_dec_set_fgs` fired 307 times, confirming that the hardware film grain register programming path is active on every frame; it is only the CPU-side grain block generation that is absent.

Unused M2M and control paths. Several thin wrappers around VB2 `ioctl`s, such as `v4l2_m2m_reqbufs`, are bypassed because the GStreamer pipeline uses active dispatch variants instead. Functions belonging to the drain and end-of-stream state machine, including `v4l2_m2m_try_schedule`, remain unused because the test pipeline terminates via file descriptor closure rather than explicit stop commands. Additional inactive paths include event-subscription and auto-cluster controls (such as `v4l2_event_subscribed`), and specific Request API cleanups like `hantro_buf_request_complete`, which are not triggered during standard streaming teardown.

4.6.3 Untraceable Functions

Of the 367 statically predicted functions, 159 could not be probed because they are absent from `available_filter_functions`. This is not a kernel configuration gap; despite enabling all necessary tracing options, the compiler inlines these functions, removing the addressable symbols required for `kprobe` attachment. These 159 missing functions span across subsystems, as a result, their runtime execution remains unverified. While specific compiler flags can force some of these functions out-of-line to enable tracing, we detail this approach in Section 5.4.

4.7 Shared Struct Access Dependencies

We apply the two complementary analysis methods described in Section 3.4 to identify which kernel structures the Hantro driver reads or writes during `ioctl` execution: syntactic field access analysis, which identifies every struct type accessed directly in the driver source code, and taint tracking, which propagates data flow from `ioctl` entry points to determine how kernel-owned structures reach the driver at runtime. The full combined results are provided in Appendix G.1.

4.7.1 Syntactic Field Access

The syntactic field access query identifies 34 kernel struct types accessed by driver code across the `ioctl`-reachable call graph, forming an upper bound on runtime kernel dependencies. Because this analysis is constrained to the call path, it maps the required structures for the active operation. These fall into four functional groups:

- **Buffer management:** `vb2_buffer`, `vb2_queue` (read/write), `vb2_v4l2_buffer` (read/write), `v4l2_m2m_buffer`—the core structures governing buffer lifecycle across VB2 and M2M.
- **Streaming context:** `v4l2_fh`, `v4l2_m2m_ctx`, `v4l2_m2m_queue_ctx`—session and queue state read by driver callbacks during streaming lifecycle events.
- **Format and capability:** `v4l2_pix_format_mplane` (read/write), `v4l2_plane_pix_format` (read/write), `v4l2_format`, `v4l2_fmtdesc`, `v4l2_frmsizeenum`, `v4l2_capability`—negotiated during setup `ioctls` and written back to user-space.
- **Control and codec payload:** `v4l2_ctrl`, `v4l2_ctrl_ptr`, `media_request_object`, and the AV1-specific nested types: `v4l2_ctrl_av1_frame`, `v4l2_ctrl_av1_sequence`, `v4l2_ctrl_av1_film_grain`, `v4l2_ctrl_av1_tile_group_entry`, `v4l2_av1_tile_info`, `v4l2_av1_quantization`, `v4l2_av1_loop_filter`, `v4l2_av1_segmentation`, `v4l2_av1_cdef`, `v4l2_av1_loop_restoration`, and `v4l2_av1_global_motion`.

4.7.2 Dataflow Taint Tracking

Running the three taint configurations described in Section 3.4.2 confirms data propagation across the kernel-driver boundary and classifies 25 structs by their data flow direction.

- **Driver struct used by the Kernel: No results.** The kernel receives driver struct pointers—for example, via `vb2_set_drv_priv`—but does not reach back into driver-owned memory within the statically traceable call graph. This is a notable insight: the kernel does not write into driver-owned state during `ioctl` execution.
- **Kernel struct used by the Driver: Confirmed.** These are the buffer lifecycle and streaming control structs passed directly as arguments to registered ops callbacks.

- **Driver queries Kernel Struct: Confirmed, but imprecise.** These are codec control structs that the driver retrieves via `hantro_get_ctrl` before dispatching to hardware. We found the results to be imprecise. Even though the sources and sinks were constrained to the active decode path as described in Section 3.3.4, the taint analysis incorrectly included structs belonging to inactive codecs (VP8, VP9, MPEG-2, H.264, and HEVC) alongside the target AV1 structs.

4.7.3 Propagation Gap of Taint Tracking

Ten AV1-specific structs appear in the syntactic results but are absent from the taint tracking data, such as `v4l2_ctrl_av1_frame`. This under-approximation gap arises from the specific way the AV1 decoder accesses control data. Rather than calling `hantro_get_ctrl` directly at each access site, the AV1 decoder caches the `v4l2_ctrl_av1_frame` pointer inside a driver-owned intermediate struct (`hantro_av1_dec_ctrls`) and dereferences nested AV1 types through it (Listing 4.9). This pattern breaks static taint tracking for two reasons: the data flow passes through driver-owned memory before arriving at the nested kernel structures, dropping the taint policy at the driver boundary; and the nested field accesses operate on an untainted base pointer. While the single-hop VP8/VP9 pattern avoids this and tracks correctly, it highlights a fatal brittleness in relying solely on dataflow analysis.

```
struct hantro_av1_dec_hw_ctx *av1_dec = &ctx->av1_dec;
struct hantro_av1_dec_ctrls *ctrls = &av1_dec->ctrls;
const struct v4l2_ctrl_av1_frame *frame = ctrls->frame;
const struct v4l2_av1_segmentation *seg = &frame->segmentation;
```

Listing 4.9: Nested struct access

Ultimately, we conclude that taint tracking is too complicated and fragile to use in practice for mapping comprehensive kernel dependencies. Realizing complete coverage requires complex data propagation rules for every edge case where pointers are cached in intermediate driver layers, all while battling the inability to cleanly prune inactive codec variations. In practice, the analysis failed to discover any missed struct accesses. In contrast, the simpler syntactic approach performed significantly better, as it was inherently bounded by the active `ioctl` call path. Consequently, it bypassed the algorithmic overhead of tracking data flow, sidestepped dead code paths belonging to inactive codecs, and captured all nested AV1 dependencies.

5 Conclusion

5.1 Summary of Contributions

This thesis addresses the question of how to systematically identify the minimal set of kernel objects required to operate the Hantro VPU driver for AV1 hardware decoding in isolation from the Linux kernel. We designed and executed a three-stage methodology: manual call graph tracing to establish ground truth, CodeQL-based static analysis to scale that understanding across all target `ioctls`, and `bpftime` dynamic tracing to confirm which statically discovered paths are actively exercised at runtime.

The analysis produced four concrete outputs. First, tracing a live GStreamer AV1 decode session with `strace` identified 15 `ioctls`, of which 11 are backed by driver-side implementations in the Hantro `v4l2_ioctl_ops` structure, forming the entry points for the static analysis (Section 4.1). Second, resolving 13 operations structures to their RK3588-specific implementations—including `hantro_queue_ops`, `rockchip_vpu981_hw_av1_dec` codec ops, and `vb2_dc_*` memory ops—constrained the call graph to the active hardware configuration; dynamic tracing confirmed these implementations at runtime (Section 4.5) and found unused function paths. Third, the cross-boundary dependency analysis identified 13 distinct interaction edges between the driver and surrounding kernel infrastructure, grouped into direct kernel dependencies (DMA, MMIO, slab allocator, runtime PM), bidirectional media framework bindings (VB2 core, `v4l2-mem2mem`, V4L2 controls), and unidirectional framework-to-driver dispatch (Section 4.4). Fourth, the struct access analysis produced 34 kernel structs accessed by the driver (Section 4.7).

5.2 Methodology Evaluation

The three-method approach proved effective because each stage compensates for the structural weaknesses of the others.

Manual tracing provided the intuition needed to scope and seed the static analysis, but it does not scale: tracing `VIDIOC_REQBUFS` alone required tracking approximately 100 functions across four subsystem layers, and extending that effort to all 11 entry points by hand would be unfeasible. Static analysis scales across the full `ioctl` set but over-approximates: without ground-truth constraints, it would have

included functions for unrelated codecs, such as H.264. The validation comparison (Section 4.3) made this concrete: resolving six discrepancy categories brought both traces into close alignment, reconciling the structural variations inherent to their respective abstraction levels.

Dynamic analysis served two roles. First, it confirmed the correctness of the static function-pointer predicates. It achieved this by verifying at runtime which `ops` struct implementations are active during a standard AV1 decode session (Section 4.5). Second, the extended trace applied to the full subsystem call graph—208 probeable functions out of 367 statically predicted—confirmed that the static analysis produces no false negatives; every function observed at runtime had been predicted, and the 44 absent functions were explained by workload constraints rather than analysis errors (Section 4.6). However, 159 statically predicted functions could not be probed at all because the kernel did not export them as traceable symbols, leaving a residual gap that our dynamic analysis cannot close.

The iterative refinement loop improved the analysis by identifying discrepancies, extending queries, and re-validating results. For example, missing `v4l2_m2m_src_buf_remove` because it was passed as a function pointer prompted us to add Case 3 to the `calls` predicate.

Dataflow taint tracking introduced complexity for little practical return. Syntactic field access alone identified all 34 kernel structures, while taint tracking failed to discover an additional dependency. Its sole contribution was distinguishing callback-injected structures ($K \rightarrow D$) from query-retrieved control structures ($DQ \rightarrow K$) to reveal how data propagates rather than what the driver depends on. Beyond this, the analysis was fragile; the AV1 pointer-caching pattern demonstrated that whenever a driver stores a kernel pointer in driver-owned state before a dereference, taint tracking drops the flow. Consequently, the simpler, call-path-constrained syntactic approach proved to be more complete, framing taint tracking as an unnecessarily complicated tool for this analysis.

5.3 Limitations

While the three-stage methodology achieved its validation goals, several factors bind the scope and precision of the results presented in Chapter 4. These fall into five groups: gaps in dynamic coverage caused by `kprobe`-based tracing itself; gaps caused by the specific decode workload exercised; missing dynamic validation of struct accesses; boundaries we drew deliberately around the analysis scope; and constraints on how far the findings generalize beyond this platform and this manual ground truth. We discuss each in turn below.

Untraceable functions. Of the 367 statically predicted functions, 159 could not be instrumented because they represent inlined functions lacking a `kprobes` entry point (Section 2.7.2). While their presence in the static result reflects the over-approximated nature of static analysis, we cannot confirm their execution. However, because these are predominantly small helper functions, verifying them is unlikely to significantly alter the overall dependency profile.

Workload coverage. By design, static analysis captures all reachable code paths, regardless of runtime conditions, creating an upper bound. Our dynamic trace covered a single, clean 307-frame AV1 decode session using `V4L2_MEMORY_MMAP`. Consequently, it left 44 statically identified functions unexercised. Code paths for the post-processor, alternative memory backends (such as `dma-buf`), film grain synthesis, end-of-stream draining, and error recovery (including `hantro_job_finish`, `hantro_return_bufs`) remain dormant without specific triggers or deliberate fault injection.

No dynamic validation of struct accesses. Section 4.6 dynamically validates the call graph, while the struct field accesses identified in Section 4.7 are not verified at runtime. The runtime execution of a function does not guarantee that every field access predicted by static analysis is actually exercised on that path.

Subsystem scope. We drew the analysis boundary tightly around the media subsystem. While the analysis identified DMA, clock, and power management dependencies as crossing edges, they were not recursively traced. Although extending the CodeQL queries to track these external dependencies is simple, doing so risks unbounded recursion, dramatically increasing analysis time and polluting the results with generic kernel utility functions.

Platform specificity. All results are tied to the RK3588 SoC running the OpenCCA 6.12 kernel. Struct layouts, compiler inlining decisions, and symbol availability naturally fluctuate across kernel releases and hardware targets. However, while the exact outputs are platform-specific, the underlying methodology is transferable; applying it to a different environment requires regenerating the CodeQL database and adapting the CodeQL library.

Manual ground-truth reliability. The manual ground-truth trace contained several human errors, including misinterpreted conditionals, a typographical error, a categorization mistake, and macro inclusions. This highlights the inherent vulnerability of manual source-code auditing and reinforces why automated cross-validation is a methodological necessity rather than merely an optional sanity check.

5.4 Future Work

The 159 untraceable functions represent an inherent limitation of `kprobe`-based dynamic analysis rather than a configuration error. Because the kernel preserves compiler-inlined optimizations, these functions lack addressable entry points despite full symbol visibility and tracking support. To circumvent this, functions declared `static inline` can be forced out-of-line by compiling the kernel with `-fno-inline`, enabling `kprobe` attachment. For `__always_inline__` functions, tools like `perf probe` [Lin26b] can resolve inlined call sites using DWARF debug information [DWA17]. This allows tracers to inject probes directly at the corresponding instruction offsets without manual source modification or offset calculation, a workflow validated in kernel tracing scenarios by Gregg [Gre14]. Building on this foundation, a promising avenue for future research is the dynamic validation of struct field accesses. The existing CodeQL library could be used to extract the source-code lines of targeted field accesses. By feeding this into DWARF-mapped `perf` probes, it could trap the CPU exactly when a field is accessed, bridging the gap between static analysis and runtime memory behavior.

Addressing the workload coverage limitation requires expanding dynamic analysis to encompass diverse operational states and fault conditions. Expanding the test suite to include bitstreams with film grain, `V4L2_MEMORY_DMABUF` buffer sharing, and explicit `V4L2_DEC_CMD_STOP` drain sequences would exercise the currently dormant feature paths, reducing the static over-approximation. Furthermore, verifying error handling and recovery dependencies demands fault injection by submitting malformed control payloads, inducing buffer starvation, or forcing hardware timeouts. Future iterations can forcefully activate dormant recovery functions (such as `hantro_job_finish`, `hantro_return_bufs`, and the `validate_av1_*` validation family). This approach will isolate the essential error-handling dependencies from safely removable dead code.

The most direct continuation of the research itself is implementing the driver extraction: constructing stubs for the shared structs and verifying that the Hantro driver completes a hardware AV1 decode job outside the Linux kernel. This would transform the analysis from a characterization of the coupling into a concrete demonstration that it can be severed.

The methodology is not specific to the Hantro driver. Applying it to other drivers, including different stateless V4L2 decoders, would test how well the CodeQL library and `bpfftrace` generator generalize. Moreover, as the manual trace was performed to bootstrap and verify the static analysis library, it would not need to be performed again.

The Python `bpfftrace` script generator and the `Hantro::CodeQL` library are reusable contributions to kernel call graph analysis. Publishing them as open-source tools would allow the community to directly benefit.

5.5 Closing Remarks

Driver code is coupled to kernel infrastructure by design: the Linux device model, VB2 buffer lifecycle, and V4L2 control framework exist to provide shared, consistent abstractions across hundreds of hardware variants. Extracting a driver from that context is non-trivial, not because of any single dependency, but because the coupling is structural and distributed across dozens of indirect call chains. This thesis contributes a concrete, reproducible methodology for quantifying that coupling by enumerating its entry points, tracing its call graph, and classifying its kernel object dependencies into actionable categories. The approach applies to any Linux subsystem where driver isolation is a goal, whether for security research, embedded rehosting, or automated exploit analysis.

List of Figures

1.1	Accessible kernel attack surface	2
2.2	Dataflow during a hardware decode request.	12
4.3	File-level call graph of VIDIOC_REQBUFS	34
4.4	Full function-level call tree of VIDIOC_REQBUFS	35
4.5	Overlay of the full driver-framework topology	36
4.6	Number of distinct reachable functions per ioctl	37

List of Tables

2.1	Comparison of Video Codec Market Share (Bitmovin Reports 8 & 9)	7
3.1	Power Supply Compatibility Observations	20
4.2	ioctls observed during the GStreamer AV1 decode run.	32
4.7	Discrepancy categories between manual and static traces.	36
4.8	Aggregated cross-boundary call edges between the driver and the non-driver files.	40
E.1	Full cross-module call-edge summary (media/driver scope)	71
E.2	Full kernel-subsystem dependency table	73
F.1	Dynamic Quantitative Execution Profile for a 307-Frame (1080p, 10s) AV1 Stream	78
G.1	Kernel structures accessed across the subsystem boundary	79
H.1	Complete static vs. dynamic function coverage diff	90

List of Listings

2.3	Polymorphism via function pointer structures in Videobuf2.	14
2.4	Example bpftrace script resolving a dynamic function pointer target.	18
3.2	CodeQL database creation with scoped kernel build	24
3.3	CodeQL predicate cases for resolving function calls	25
3.4	bpftrace example for resolving function pointers via BTF	30
3.5	Example of a generated kprobe for invocation counting	30
4.1	Pipeline to execute hardware-accelerated AV1 decode	32
4.9	Nested struct access	46
B.1	Kernel configuration flags for debugging and tracing	65
B.2	Full .clangd configuration for ARM64 kernel development	66
C.1	Recursive Function Calls Query	67
C.2	Query to identify untracked function pointer implementations	67
C.3	Syntactic struct access query	68

Bibliography

- [AAFX21] Muhammad Abubakar, Adil Ahmad, Pedro Fonseca, and Dongyan Xu: *SHARD: Fine-grained kernel specialization with context-aware hardening*. In *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, 2021.
- [Bit25a] Bitmovin Inc.: *8th annual video developer report 2024/2025*. Research report, Bitmovin, February 2025. <https://bitmovin.com/video-developer-report/>, visited on 2026-03-25, Insights from 167 video developers across 34 countries.
- [Bit25b] Bitmovin Inc.: *9th annual video developer report 2025/2026*. Research report, Bitmovin, September 2025. <https://bitmovin.com/video-developer-report/>, visited on 2026-03-25, Presented at IBC 2025.
- [BS25] Andrin Bertschi and Shweta Shinde: *Opencca: An open framework to enable arm cca research*. In *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 429–434, 2025.
- [BWC⁺21] Benjamin Bross, Ye Kui Wang, Shanshe Cao, *et al.*: *Overview of the versatile video coding (VVC) standard and its licensing landscape compared to HEVC*. *IEEE Transactions on Circuits and Systems for Video Technology*, 31(10):3736–3764, 2021.
- [CdM23] Arnaldo Carvalho de Melo: *Dwarves and pahole struct layout optimization utilities*, 2023. <https://git.kernel.org/pub/scm/devel/pahole/pahole.git>.
- [CMHea20] Yue Chen, Debargha Mukherjee, Jingning Han, and *et al.*: *An overview of coding tools in av1: the first video codec from the alliance for open media*. *APSIPA Transactions on Signal and Information Processing*, 9, January 2020.
- [CRKH05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman: *Linux device drivers, third edition*. O’Reilly Media, Inc., 3rd edition, 2005, ISBN 9780596005900.
- [Duf20] Nicolas Dufresne: *Linux stateless video decoder support*, 2020.

- [DWA17] DWARF Standards Committee: *DWARF debugging information format, version 5*. Technical report, DWARF Standards Committee, 2017. <https://dwarfstd.org/doc/DWARF5.pdf>.
- [EBSA⁺12] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger: *Dark silicon and the end of multicore scaling*. IEEE Micro, 32(3):122–134, 2012.
- [Ern03] Michael D. Ernst: *Static and dynamic analysis: Synergy and duality*. In *Proceedings of the 1st International Workshop on Dynamic Analysis (WODA)*, pages 24–27. IEEE, 2003.
- [FFD⁺25] Erhu Feng, Dahu Feng, Dong Du, Yubin Xia, and Haibo Chen: *snpu: Trusted execution environments on integrated npus*. In *Proceedings of the 51st Annual International Symposium on Computer Architecture, ISCA '24*, page 708–723. IEEE Press, 2025, ISBN 9798350326581. <https://doi.org/10.1109/ISCA59077.2024.00057>.
- [Git24] GitHub, Inc.: *Codeql semantic code analysis engine toolchain*, 2024. <https://codeql.github.com/>.
- [Git26a] GitHub: *Codeql documentation: Ql language reference*, 2026. <https://codeql.github.com/docs/ql-language-reference/>.
- [Git26b] GitHub: *Codeql library for c and c++*, 2026. <https://codeql.github.com/docs/codeql-language-guides/codeql-library-for-cpp/>.
- [Gre14] Brendan Gregg: *Kernel line tracing: Linux perf rides the rocket*. Brendan Gregg’s Blog, September 2014. <https://www.brendangregg.com/blog/2014-09-11/perf-kernel-line-tracing.html>.
- [Gre19] Brendan Gregg: *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019, ISBN 978-0136554820.
- [GSt26a] GStreamer Developers: *GStreamer v4l2codecs Plugin Documentation*. freedesktop.org, 2026. <https://gstreamer.freedesktop.org/documentation/v4l2codecs/index.html>.
- [GSt26b] GStreamer Project: *GStreamer open source multimedia framework*, 2026. <https://gstreamer.freedesktop.org>.
- [HHE⁺24] Yongzhe Huang, Kaiming Huang, Matthew Ennis, Vikram Narayanan, Anton Burtsev, Trent Jaeger, and Gang Tan: *SoK: Understanding the Attack Surface in Device Driver Isolation Frameworks*. December 2024.
- [HM10] Tejun Heo and Florian Mickler: *Workqueue*. Linux Kernel Organization, 2010. <https://www.kernel.org/doc/html/v6.12/core-api/workqueue.html>.

- [HND⁺22] Yongzhe Huang, Vikram Narayanan, David Detweiler, Kaiming Huang, Gang Tan, Trent Jaeger, and Anton Burtsev: *KSplit: Automating device driver isolation*. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 613–631. USENIX Association, 2022.
- [JSF26] Seth Jenkins, Natalie Silvanovich, and Ivan Fratric: *A 0-click exploit chain for the pixel 9: Decoding dolby*. Google Project Zero Security Research Blog, January 2026. <https://projectzero.google/>.
- [Lin25] Linux Manual Pages: *ioctl(2) - Linux manual page*. Linux Kernel Organization, 2025. <https://man7.org/linux/man-pages/man2/ioctl.2.html>.
- [Lin26a] Linux Manual Pages: *mmap(2) - Linux manual page*. Linux Kernel Organization, 2026. <https://man7.org/linux/man-pages/man2/mmap.2.html>.
- [Lin26b] Linux Manual Pages: *perf-probe(1) — Linux manual page*. Linux Kernel Organization, 2026. <https://man7.org/linux/man-pages/man1/perf-probe.1.html>.
- [Lin26c] Linux Manual Pages: *proc_kallsyms(5) - Linux manual page*. The Linux Kernel Organization, 2026. https://man7.org/linux/man-pages/man5/proc_kallsyms.5.html.
- [LLV24] LLVM Project: *Clangd Documentation*, 2024. <https://clangd.llvm.org/>, Version 18.1.
- [Mak21] Slava Makkaveev: *Looking for vulnerabilities in MediaTek audio DSP*, November 2021. <https://research.checkpoint.com/2021/looking-for-vulnerabilities-in-mediatek-audio-dsp/>, visited on 2026-06-02.
- [MDS⁺24] Lukas Maar, Florian Draschbacher, Lorenz Schumm, Ernesto Martínez García, and Stefan Mangard: *The doom of device drivers: Your android device (most likely) has N-Day kernel vulnerabilities*. In *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024.
- [MPK⁺06] Ananth Mavinakayanahalli, Prasanna Panchamukhi, Jim Keniston, Anil Keshavamurthy, and Masami Hiramatsu: *Probing the guts of kprobes*. In *Proceedings of the Linux Symposium*, volume 2, pages 109–124, 2006. <https://www.kernel.org/doc/ols/2006/ols2006v2-pages-109-124.pdf>.
- [MRR04] Ana Milanova, Atanas Rountev, and Barbara G Ryder: *Precise call graph construction for C programs with function pointers*. Automated Software Engineering, 11(1), 2004.

- [MS18] Anders Møller and Michael I. Schwartzbach: *Static program analysis*, October 2018. Department of Computer Science, Aarhus University, <http://cs.au.dk/~amoeller/spa/>.
- [Nat22] National Vulnerability Database: *CVE-2022-22087 Detail*, 2022. <https://nvd.nist.gov/vuln/detail/cve-2022-22087>.
- [NB⁺19] Vikram Narayanan, Abhiram Balasubramanian, Charlie Jacobsen, Sarah Spall, Scott Bauer, Michael Quigley, Aftab Hussain, Abdullah Younis, Junjie Shen, Moinak Bhattacharyya, and Anton Burtsev: *LXDs: Towards isolation of kernel subsystems*. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 269–284. USENIX Association, 2019.
- [Neo24] NeoVim Developer Community: *Neovim: Hyperextensible vim-based text editor*, 2024. <https://neovim.io/>.
- [NNH99] Flemming Nielson, Hanne Nielson, and Chris Hankin: *Principles of Program Analysis*. January 1999, ISBN 978-3-642-08474-4.
- [Par25] N Parthiban: *Demystifying the embedded linux graphics stack*, 2025.
- [Rad26a] Radxa Computer Co., Ltd.: *ROCK 5B Kernel Development*, 2026. <https://docs.radxa.com/en/rock5/rock5b/low-level-dev/kernel>.
- [Rad26b] Radxa Computer Co., Ltd.: *ROCK 5B Product Documentation*, 2026. <https://docs.radxa.com/en/rock5/rock5b>.
- [Ric04] I.E. Richardson: *H.264 and MPEG-4 Video Compression: Video Coding for Next-generation Multimedia*. Wiley, 2004, ISBN 9780470869604. <https://books.google.ch/books?id=n9YVhx2zgZ4C>.
- [Roc22] Rockchip Electronics Co., Ltd.: *Rockchip rk3588 technical reference manual (trm)*. Technical report, stanford, March 2022. https://www.scs.stanford.edu/~zyedidia/docs/rockchip/rk3588_part1.pdf, Revision 1.0, Part 1.
- [Roc24] Rockchip Open Source Community: *Rockchip FIQ Debugger User Guide*. Rockchip Electronics Co., Ltd., June 2024. <https://github.com/as-jackson/Rockchip-Docs>, Revision V1.2.0.
- [Ros24] Steven Rostedt: *Ftrace - Function Tracer*. The Linux Kernel Documentation, 2024. <https://www.kernel.org/doc/html/v6.12/trace/ftrace.html>.
- [RV24] Rockchip and VeriSilicon: *Hantro VPU media driver*. Linux kernel source, [drivers/media/platform/verisilicon/](https://github.com/torvalds/linux/tree/v6.12/drivers/media/platform/verisilicon/), 2024. <https://github.com/torvalds/linux/tree/v6.12/drivers/media/platform/verisilicon>.

- [Sil26] Natalie Silvanovich: *A 0-click exploit chain for the pixel 9 part 1: Decoding dolby*, 2026. <https://projectzero.google/2026/01/pixel-0-click-part-1.html>, visited on 2026-03-21.
- [Str24] Strace Development Team: *Strace: The diagnostic, debugging and instructional userspace utility for linux*, 2024. <https://strace.io/>.
- [Tha25] Dave Thaler: *Bpf instruction set architecture (isa)*. Internet-draft, IETF BPF Working Group, 2025. <https://datatracker.ietf.org/doc/html/draft-ietf-bpf-isa>.
- [The24a] The Linux Kernel Developers: *BPF Type Format (BTF)*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/bpf/btf.html>.
- [The24b] The Linux Kernel Developers: *Common Clock Framework*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/clock.html>.
- [The24c] The Linux Kernel Developers: *Contiguous Memory Allocator (CMA) Debugfs Interface*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/admin-guide/mm/cma_debugfs.html.
- [The24d] The Linux Kernel Developers: *Devres - Managed Device Resources*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/devres.html>.
- [The24e] The Linux Kernel Developers: *Dynamic DMA Mapping Guide*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/core-api/dma-api.html>.
- [The24f] The Linux Kernel Developers: *eBPF Documentation*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/bpf/index.html>.
- [The24g] The Linux Kernel Developers: *Event Interface*. Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/userspace-api/media/v4l/dev-event.html>.
- [The24h] The Linux Kernel Developers: *ioctl VIDIOC_REQBUFS*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/userspace-api/media/v4l/vidioc-reqbufs.html>.
- [The24i] The Linux Kernel Developers: *Kernel Probes (Kprobes)*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/trace/kprobes.html>.
- [The24j] The Linux Kernel Developers: *The Linux Kernel API*. Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/core-api/kernel-api.html>.

- [The24k] The Linux Kernel Developers: *Linux Kernel Driver Model*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/>.
- [The24l] The Linux Kernel Developers: *Media Controller Request API*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/userspace-api/media/mediactl/request-api.html>.
- [The24m] The Linux Kernel Developers: *Memory Allocation Guide*. Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/core-api/memory-allocation.html>.
- [The24n] The Linux Kernel Developers: *Memory-to-memory Stateless Video Decoder Interface*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/userspace-api/media/v4l/dev-stateless-decoder.html>.
- [The24o] The Linux Kernel Developers: *pin_user_pages() and related calls*. Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/core-api/pin_user_pages.html.
- [The24p] The Linux Kernel Developers: *Platform Devices and Drivers*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/platform.html>.
- [The24q] The Linux Kernel Developers: *Runtime Power Management Framework*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/power/runtime_pm.html.
- [The24r] The Linux Kernel Developers: *V4L2 Memory-to-Memory Functions and Data Structures*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-mem2mem.html>.
- [The24s] The Linux Kernel Developers: *Video4Linux Framework*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-core.html>.
- [The24t] The Linux Kernel Developers: *VideoBuffer2 (vb2) Framework*. The Linux Kernel Organization, 2024. <https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-videobuf2.html>.
- [The26] The Linux Kernel Developers: *include/uapi/linux/v4l2-controls.h: AV1 stateless control definitions*, 2026. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/v4l2-controls.h>.

- [VCS23] Willy R. Vasquez, Stephen Checkoway, and Hovav Shacham: *The most dangerous codec in the world: Finding and exploiting vulnerabilities in h.264 decoders*. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6647–6664, Anaheim, CA, August 2023. USENIX Association, ISBN 978-1-939133-37-3. <https://www.usenix.org/conference/usenixsecurity23/presentation/vasquez>.
- [Ver24] VeriSilicon: *Hantro VPU IP*, 2024. <https://www.verisilicon.com/en/IPPortfolio/HantroVPUIP>.
- [W⁺25] Meng Wu *et al.*: *Statically discover cross-entry use-after-free vulnerabilities in the linux kernel*. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.
- [WLW⁺17] Zhen Wang, Xi Li, Chao Wang, Zhinan Cheng, Jiachen Song, and Xuehai Zhou: *Rethinking energy-efficiency of heterogeneous computing for cnn-based mobile applications*. In *2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications (ISPA/IUCC)*, pages 454–458, 2017.
- [ZBFB14] Jonas Zaddach, Luca Bruno, Aurélien Francillon, and Davide Balzarotti: *Avatar: A framework to support dynamic security analysis of embedded systems’ firmwares*. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.

A Research Artifacts

All source files, query outputs, and raw results produced during this thesis are archived in a GitLab repository at https://gitlab.inf.ethz.ch/PRV-SHINDE/theses/ba-theses/ba-2026/ba_till_begue/Bachelor_Thesis.git. The repository is organized as follows:

- manual_trace_reqbufs.md** The complete manual call graph trace for `VIDIOC_REQBUFS`, serving as the ground truth used to bootstrap and validate the static analysis (Section 3.2).
- ql_queries/** All CodeQL queries and the `hantro_lib` library developed during this work, including the recursive call graph query, the struct access queries, and the Discovery query used to drive automated probe generation. Raw query outputs are provided in `ql_queries/query_output/`.
- bpf_trace/** The Python `bpftrace` script generator (`gen_bpf.py`), the extended generator script (`gen_bpf2.py`) and all generated probe scripts, together with the raw `bpftrace` output collected during the 307-frame AV1 decode session on the Rock 5B.
- strace_output/** The raw `strace` logs captured during the GStreamer decode session, used to identify the `ioctl` entry points analyzed in Chapter 3.
- graphs/** Call graph visualizations generated from the CodeQL output, including the layered architecture diagram and the annotated `VIDIOC_REQBUFS` call graph.

B Environment Configuration

B.1 Kernel Config File

This was appended to the `/bsp/linux/rk2410/0001-common/kconfig.conf` file.

```
# Debugging and Symbol Support
CONFIG_DEBUG_INFO=y
CONFIG_DEBUG_INFO_DWARF5=y
CONFIG_DEBUG_INFO_BT=y
CONFIG_DEBUG_INFO_BT_MODULES=y
CONFIG_PAHOLE_HAS_SPLIT_BT=y
CONFIG_FRAME_POINTER=y
CONFIG_KALLSYMS=y
CONFIG_KALLSYMS_ALL=y

# BPF and Tracing Infrastructure
CONFIG_BPF=y
CONFIG_BPF_EVENTS=y
CONFIG_KPROBES=y
CONFIG_KPROBE_EVENTS=y
CONFIG_TRACING=y
CONFIG_FTRACE=y
CONFIG_FUNCTION_TRACER=y
CONFIG_FUNCTION_GRAPH_TRACER=y
CONFIG_DYNAMIC_FTRACE=y
CONFIG_FTRACE_MCOUNT_RECORD=y
CONFIG_TRACEPOINTS=y
CONFIG_TRACEFS=y

# Ethernet and Driver Support
CONFIG_STMMAC_ETH=y
CONFIG_DWMAC_ROCKCHIP=y
```

Listing B.1: Kernel configuration flags for debugging and tracing

B.2 Clangd Configuration File (.clangd)

```
CompileFlags:
  Add:
    - --target=aarch64-linux-gnu
  Remove:
    # GCC-only arch flags
    - -mabi=lp64
    - -march=armv8-a
    - -mrecord-mcount
    - -mno-outline-atomics
    # GCC-only optimization/codegen flags
    - -fconserve-stack
    - -fno-allow-store-data-races
    - -fno-var-tracking-assignments
    - -fno-ipa-sra
    - --param=allow-store-data-races=0
    # GCC-only warnings
    - -Wno-unused-but-set-variable
    - -Wno-maybe-uninitialized
    - -Wno-packed-not-aligned
    - -Wno-format-truncation
    - -Wno-format-overflow
    - -Wno-frame-address
    - -Wno-alloc-size-larger-than

Diagnostics:
  Suppress:
    - drv_unknown_argument
```

Listing B.2: Full .clangd configuration for ARM64 kernel development

C CodeQL Queries

```
import cpp
import hantro_lib

from Function entry, Function caller, Function reached, string section
where
  Hantro::isV4l2Ioctl(entry) and
  Hantro::calls*(entry, caller) and
  Hantro::calls(caller, reached) and
  Hantro::isSubsystemFile(caller.getFile()) and
  not reached.getName().matches("%compiletime_assert%") and
  (
    Hantro::isDriverFile(reached.getFile()) and section = "1_Driver"
    or
    Hantro::isSubsystemFile(reached.getFile()) and section = "2_SubSystem"
    or
    not Hantro::isSubsystemFile(reached.getFile()) and section = "3_kernel"
  )
select
  section                                as scope,
  caller.getName()                       as caller_func,
  reached.getName()                      as called_func,
  reached.getFile().getBaseName() as file,
  concat(entry.getName(), ", ") as ioctl_list
order by scope
```

Listing C.1: Recursive Function Calls Query

```
import cpp
import hantro_lib

from Function entry, ExprCall c, Field f, Function callee
where
  Hantro::isV4l2Ioctl(entry) and
  Hantro::callsUnfiltered*(entry, c.getEnclosingFunction()) and
  Hantro::resolvesFunctionPointerCall(c, f, callee) and
  not Hantro::isTrackedOpsStruct(Hantro::fieldStructName(f))
select
  f.getDeclaringType().getName() as ops_struct,
  f.getName() as field,
  c.getEnclosingFunction().getName() as called_from,
  c.getFile().getBaseName() as file,
  c.getLocation().getStartLine() as line,
  callee.getName() as implementation_func
order by ops_struct, field
```

Listing C.2: Query to identify untracked function pointer implementations

```

import cpp
import hantro_lib

string getDomain(File f) {
  if Hantro::isDriverFile(f) then result = "Driver" else result = "Kernel"
}

Struct getStructType(FieldAccess fa) {
  result = fa.getTarget().getDeclaringType().getUnspecifiedType()
}

string getAccessType(Function f, Struct s) {
  // Ensure the struct is actually accessed in this function
  exists(FieldAccess fa | fa.getEnclosingFunction() = f and getStructType(fa) = s) and

  if exists(FieldAccess fa |
    fa.getEnclosingFunction() = f and
    getStructType(fa) = s and
    fa.isModified()
  )
  then result = "Modified"
  else result = "Read-only"
}

from
  Function entry,
  Function reached,
  Struct s,
  string direction,
  string accessType
where
  Hantro::isV4l2Ioctl(entry) and
  Hantro::calls*(entry, reached) and
  not s.getName() = "" and

  exists(string funcDomain, string structDomain |
    funcDomain = getDomain(reached.getFile()) and
    structDomain = getDomain(s.getFile()) and
    funcDomain != structDomain and
    direction = funcDomain + " -> " + structDomain
  ) and
  accessType = getAccessType(reached, s)

select
  s.getName() as struct_name,
  accessType as access_type

```

Listing C.3: Syntactic struct access query

D Discrepancy Taxonomy

Category 1: Conditional Check Misinterpretation (Manual Error) Representative functions: `fh_to_ctx`, `vidioc_g_fmt_out_mplane`

During manual tracing, conditional checks of the form `if (ops->function)` were misinterpreted as function invocations. These checks test for the existence of a function pointer to determine whether optional functionality is available, but do not execute. Static analysis correctly identified these as data accesses and excluded them.

Category 2: Static Analysis False Negative (Query Gap) Function: `v4l2_m2m_src_buf_remove`

This function was passed as a function pointer argument to `hantro_return_bufs(q, v4l2_m2m_src_buf_remove)`, where it is stored and invoked indirectly. The initial `calls` predicate only captured direct calls and `ops` struct invocations, missing this higher-order function pattern. We resolved this by adding Case 3 to the `calls` predicate (Listing 3.3), which identifies functions passed by reference as arguments.

Category 3: Typographical Errors Functions: `v4l2_ioctl_ops`, `v4l_reqbuf`

`v4l2_ioctl_ops` is a structure, not an executable function. We included it in the manual trace due to its central role in resolving the first indirect call, but incorrectly classified it as a callable function. `v4l_reqbuf` was a simple typographical error; the correct function name is `v4l_reqbufs` (with a trailing 's'). Static analysis correctly excluded both.

Category 4: Preprocessor Macros Representative functions: `container_of`, `ARRAY_SIZE`, `WARN_ON`, `is_valid_ioctl`

Preprocessor macros are source-level text substitutions that the C compiler expands before compilation, meaning they never appear as callable functions in the compiled binary; CodeQL operates on the Abstract Syntax Tree (AST) produced after this expansion and therefore generates no `FunctionCall` edges for macro invocations. The

manual trace captured these as human-readable developer semantics, whereas CodeQL correctly reflects compilation reality. These were filtered from the comparison as an inherent difference between the two representations.

Category 5: subsystem Aliasing Representative functions: `dev_err` vs. `_dev_err`, `kzalloc` vs. `kzalloc_noprof`

The Linux kernel aliases many base implementations for logging and debugging. The manual trace recorded source-level API names (e.g., `dev_err`, `kzalloc`), while static analysis resolved down to their compiled targets (e.g., `_dev_err`, `kzalloc_noprof`). This is not a logical discrepancy but an inherent difference in the abstraction level between human and compiler perspectives. We noted these differences without attempting to unify them.

Category 6: Hardware Scope and Active Implementation Representative functions: `hantro_h264_dec_exit`, `vb2_vmalloc_alloc`, `vb2_dma_sg_alloc`

The manual trace was implicitly constrained to the active RK3588 AV1 hardware configuration. Static analysis traversed all function pointer initializations in the driver, including routines for unrelated codecs (H.264, VP8) and alternative VB2 memory backends (vmalloc, dma-sg) that are never active on this platform. Identifying the correct active implementations for each `ops` structs was precisely the goal of the iterative filtering process described in Section 3.3.4, and we confirmed the resulting assumptions through dynamic analysis (Section 3.5). Inactive branches were pruned from the static output to match the active hardware configuration.

E Cross-Module Call Graph

Table E.1: Full cross-module call-edge summary (media/driver scope). Each row is one aggregated (caller module, callee module) pair within the media subsystem and driver. Max reach: highest ioctl count on any single raw row. Ioctls (union): all unique ioctls observed for that pair.

Caller	Callee	Max reach	Ioctls (union)
V4L2 ioctl dispatch	V4L2 core (dev/common)	7	v4l_dqbuf, v4l_enum_fmt, v4l_g_fmt, v4l_qbuf, v4l_query_ext_ctrl, v4l_querybuf, v4l_querycap, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt
V4L2 controls	Media controller	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 mem2mem	VB2 core	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
VB2 core	Media controller	4	v4l_dqbuf, v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
VB2 core	VB2 DMA-contig alloc	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
Hantro generic	V4L2 core (dev/common)	4	v4l_qbuf, v4l_querycap, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamon
Hantro generic	V4L2 mem2mem	4	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon

Continued on next page

Table E.1 – *continued*

Caller	Callee	Max reach	Ioctl's (union)
Hantro generic	VB2 core	4	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon
V4L2 controls	V4L2 event	3	v4l_qbuf, v4l_s_ext_ctrls, v4l_streamon
V4L2 controls	Hantro generic	3	v4l_qbuf, v4l_s_ext_ctrls, v4l_streamon
V4L2 mem2mem	Hantro generic	2	v4l_qbuf, v4l_streamon
VB2 core	Hantro generic	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
VB2 DMA-contig alloc	VB2 core	2	v4l_qbuf, v4l_streamon
VB2 DMA-contig alloc	frame_vector	2	v4l_qbuf, v4l_reqbufs
Hantro generic	V4L2 controls	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro generic	V4L2 event	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro generic	VB2 DMA-contig alloc	2	v4l_qbuf, v4l_streamon
Hantro generic	Hantro RK AV1	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro RK AV1	VB2 core	2	v4l_qbuf, v4l_streamon
Hantro RK AV1	VB2 DMA-contig alloc	2	v4l_qbuf, v4l_streamon
Hantro RK AV1	Hantro generic	2	v4l_qbuf, v4l_streamon
V4L2 ioctl dispatch	V4L2 mem2mem	1	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
V4L2 ioctl dispatch	V4L2 controls	1	v4l_query_ext_ctrl, v4l_s_ext_ctrls
V4L2 ioctl dispatch	Media controller	1	v4l_querycap, v4l_s_fmt
V4L2 ioctl dispatch	Hantro generic	1	v4l_enum_fmt, v4l_g_fmt, v4l_querycap, v4l_s_fmt, v4l_stub_enum_framesizes
V4L2 mem2mem	V4L2 core (dev/common)	1	v4l_qbuf

Continued on next page

Table E.1 – *continued*

Caller	Callee	Max reach	Ioctl's (union)
V4L2 controls	V4L2 core (dev/common)	1	v4l_s_ext_ctrls

Table E.2: Full kernel-subsystem dependency table. Each row shows one (media-layer caller module, kernel subsystem group) pair. Kernel subsystem groups are formed by collapsing individual kernel header files into functional categories (e.g. all spinlock and mutex headers into “Locking”).

Caller layer)	(media Kernel subsystem	Max reach	Ioctl's (union)
V4L2 core (dev/common)	VFS / file ops	10	v4l_dqbuf, v4l_enum_fmt, v4l_g_fmt, v4l_qbuf, v4l_query_ext_ctrl, v4l_querybuf, v4l_querycap, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt
V4L2 controls	List / bitops / atomics	6	v4l_dqbuf, v4l_qbuf, v4l_query_ext_ctrl, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 controls	Locking	6	v4l_dqbuf, v4l_qbuf, v4l_query_ext_ctrl, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
VB2 core	List / bitops / atomics	6	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
V4L2 controls	Memory mgmt	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 controls	Printk / string / error	5	v4l_qbuf, v4l_query_ext_ctrl, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 event	List / bitops / atomics	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon

Continued on next page

Table E.2 – *continued*

Caller layer)	(media	Kernel subsystem	Max reach	Ioctls (union)
V4L2 event		Locking	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 event		Timekeeping	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 event		Wait queue / scheduling	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 mem2mem		List / bitops / atomics	4	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
V4L2 mem2mem		Locking	4	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
VB2 core		Locking	4	v4l_dqbuf, v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
VB2 core		Printk / string / error	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
VB2 core		Wait queue / scheduling	4	v4l_dqbuf, v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
VB2 DMA-contig alloc		DMA / scatter-gather / DMABUF	4	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
VB2 DMA-contig alloc		Memory mgmt	4	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
VB2 DMA-contig alloc		Reference counting	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_stub_expbuf
Hantro generic		Printk / string / error	4	v4l_g_fmt, v4l_qbuf, v4l_querycap, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamon, v4l_stub_enum_framesizes

Continued on next page

Table E.2 – *continued*

Caller layer)	(media	Kernel subsystem	Max reach	Ioctls (union)
Hantro RK AV1		DMA / scatter-gather / DMABUF	4	v4l_qbuf, v4l_reqbufs, v4l_ streamoff, v4l_streamon
V4L2 core (dev/common)		Timekeeping	3	v4l_dqbuf, v4l_qbuf, v4l_ querybuf
V4L2 mem2mem		Printk / string / error	2	v4l_qbuf, v4l_streamoff, v4l_ streamon
V4L2 mem2mem		Wait queue / scheduling	2	v4l_qbuf, v4l_streamoff, v4l_ streamon
VB2 core		DMA / scatter-gather / DMABUF	2	v4l_qbuf, v4l_reqbufs, v4l_ stub_expbuf
VB2 DMA-contig alloc		Printk / string / error	2	v4l_qbuf, v4l_reqbufs, v4l_ stub_expbuf
MM frame_vector		Memory mgmt	2	v4l_qbuf, v4l_reqbufs
MM frame_vector		Printk / string / error	2	v4l_qbuf, v4l_reqbufs
Hantro generic		DMA / scatter-gather / DMABUF	2	v4l_qbuf, v4l_reqbufs, v4l_ streamoff, v4l_streamon
Hantro generic		MMIO	2	v4l_qbuf, v4l_streamon
Hantro generic		Power mgmt / clock	2	v4l_qbuf, v4l_streamon
Hantro generic		Wait queue / scheduling	2	v4l_qbuf, v4l_streamon
Hantro RK AV1		Memory mgmt	2	v4l_qbuf, v4l_streamon
Hantro RK AV1		Printk / string / error	2	v4l_qbuf, v4l_streamon
V4L2 ioctl dispatch		List / bitops / atomics	1	v4l_query_ext_ctrl, v4l_ reqbufs, v4l_s_ext_ctrls
V4L2 ioctl dispatch		Printk / string / error	1	v4l_enum_fmt, v4l_g_fmt, v4l_ reqbufs, v4l_s_fmt
V4L2 core (dev/common)		Device model	1	v4l_querycap, v4l_s_ext_ ctrls
V4L2 controls		User-space access	1	v4l_s_ext_ctrls
Media controller		Locking	1	v4l_s_fmt
VB2 core		Memory mgmt	1	v4l_reqbufs

Continued on next page

Table E.2 – *continued*

Caller layer)	(media	Kernel subsystem	Max reach	Ioctls (union)
VB2 DMA-contig alloc		Device model	1	v4l_reqbufs
MM frame_vector		List / bitops / atomics	1	v4l_qbuf

F ops Struct Dynamic Results

Table F.1 details the dynamic execution status and precise invocation counts of the hardware-specific implementation functions mapped during the static analysis of the Rockchip RK3588 AV1 decoding pipeline. The empirical workload was driven by a 10-second, 1080p AV1 clip (Sintel_1080_10s_10MB.mp4) containing exactly 307 encoded frames.

Function Name	Ops Struct	Count
Fired (Confirmed Execution Path)		
vb2_dc_cookie	vb2_mem_ops	2,679
__fill_v4l2_buffer	vb2_buf_ops	1,242
vb2_dc_num_users	vb2_mem_ops	1,242
v4l2_ctrl_type_op_validate	v4l2_ctrl_type_ops	923
v4l2_ctrl_type_op_equal	v4l2_ctrl_type_ops	922
__init_vb2_v4l2_buffer	vb2_buf_ops	628
__copy_timestamp	vb2_buf_ops	614
__fill_vb2_buffer	vb2_buf_ops	614
__verify_planes_array_core	vb2_buf_ops	614
hantro_buf_out_validate	vb2_ops	614
hantro_buf_prepare	vb2_ops	614
hantro_buf_queue	vb2_ops	614
v4l2_m2m_ioctl_dqbuf	v4l2_ioctl_ops	614
v4l2_m2m_ioctl_qbuf	v4l2_ioctl_ops	614
vb2_dc_finish	vb2_mem_ops	614
vb2_dc_prepare	vb2_mem_ops	614
device_run	v4l2_m2m_ops	307
rockchip_vpu981_av1_dec_done	hantro_codec_ops	307
rockchip_vpu981_av1_dec_run	hantro_codec_ops	307
rockchip_vpu981_postproc_disable	Optional HW Feature	307
v4l2_ctrl_request_unbind	media_request_object_ops	307
vb2_req_unbind	media_request_object_ops	307
vb2_dc_put	vb2_mem_ops	42
v4l2_ctrl_type_op_init	v4l2_ctrl_type_ops	31
v4l2_m2m_ioctl_expbuf	v4l2_ioctl_ops	14
v4l2_m2m_ioctl_querybuf	v4l2_ioctl_ops	14
vb2_dc_alloc	vb2_mem_ops	14

Function Name	Ops Struct	Count
vb2_dc_get_dmabuf	vb2_mem_ops	14
vidioc_enum_fmt_vid_cap	v4l2_ioctl_ops	12
vidioc_enum_framesizes	v4l2_ioctl_ops	12
vidioc_g_fmt_cap_mplane	v4l2_ioctl_ops	6
vidioc_enum_fmt_vid_out	v4l2_ioctl_ops	6
vidioc_s_fmt_out_mplane	v4l2_ioctl_ops	5
v4l2_m2m_ioctl_reqbufs	v4l2_ioctl_ops	4
hantro_av1_s_ctrl	v4l2_ctrl_ops	3
hantro_try_ctrl	v4l2_ctrl_ops	3
vidioc_querycap	v4l2_ioctl_ops	3
hantro_queue_setup	vb2_ops	2
hantro_start_streaming	vb2_ops	2
hantro_stop_streaming	vb2_ops	2
v4l2_m2m_ioctl_streamoff	v4l2_ioctl_ops	2
v4l2_m2m_ioctl_streamon	v4l2_ioctl_ops	2
rockchip_vpu981_av1_dec_exit	hantro_codec_ops	1
rockchip_vpu981_av1_dec_init	hantro_codec_ops	1
Mapped in Static Analysis, Not Fired dynamically		
hantro_buf_request_complete	vb2_ops	0 (Not Fired)
rockchip_vpu981_postproc_enable	Optional HW Feature	0 (Not Fired)
vb2_dc_attach_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_detach_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_get_userptr	vb2_mem_ops	0 (Not Fired)
vb2_dc_map_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_put_userptr	vb2_mem_ops	0 (Not Fired)
vb2_dc_unmap_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_ops_wait_finish	vb2_ops	0 (Not Fired)
vb2_ops_wait_prepare	vb2_ops	0 (Not Fired)

Table F.1: Dynamic Quantitative Execution Profile for a 307-Frame (1080p, 10s) AV1 Stream

G Shared Structs

Table G.1: All kernel structures accessed across the subsystem boundary, combining syntactic field access and taint tracking results. Access type reflects the union of both methods. Flow direction follows the taint tracking classification: $K \rightarrow D$ = kernel injects into driver via callback; $DQ \rightarrow K$ = driver queries kernel via `hantro_get_ctrl`; $K \leftrightarrow D$ = bidirectional access observed.

Struct	Flow	Access	Method
<i>Kernel $\rightarrow$ Driver (callback-injected)</i>			
<code>device</code>	$K \rightarrow D$	Read	Both
<code>device_driver</code>	$K \rightarrow D$	Read	Both
<code>media_request_object</code>	$K \rightarrow D$	Read	Both
<code>v4l2_capability</code>	$K \rightarrow D$	Read	Both
<code>v4l2_fh</code>	$K \rightarrow D$	Read	Both
<code>v4l2_fmtdesc</code>	$K \rightarrow D$	Read/Write	Both
<code>v4l2_format</code>	$K \rightarrow D$	Read	Both
<code>v4l2_frmsizeenum</code>	$K \rightarrow D$	Read/Write	Both
<code>v4l2_m2m_ctx</code>	$K \rightarrow D$	Read	Both
<code>v4l2_m2m_queue_ctx</code>	$K \rightarrow D$	Read	Both
<code>v4l2_pix_format_mplane</code>	$K \rightarrow D$	Read/Write	Both
<code>v4l2_plane_pix_format</code>	$K \rightarrow D$	Read/Write	Both
<code>video_device</code>	$K \rightarrow D$	Read	Both
<i>Driver Query $\rightarrow$ Kernel (query-retrieved)</i>			
<code>v4l2_ctrl</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_ptr</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_av1_sequence</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_av1_film_grain</code>	$DQ \rightarrow K$	Read	Syntactic
<code>v4l2_ctrl_av1_frame</code>	$DQ \rightarrow K$	Read	Syntactic
<code>v4l2_ctrl_h264_sps</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_hevc_sps</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_vp9_frame</code>	$DQ \rightarrow K$	Read	Both
<code>v4l2_ctrl_vp9_compressed_hdr</code>	$DQ \rightarrow K$	Read	Taint
<code>v4l2_ctrl_mpeg2_picture</code>	$DQ \rightarrow K$	Read	Taint

Continued on next page

Table G.1 (*continued*)

Struct	Flow	Access	Method
v4l2_ctrl_mpeg2_quantisation	DQ→K	Read	Taint
v4l2_ctrl_mpeg2_sequence	DQ→K	Read	Taint
v4l2_h264_reflist_builder	DQ→K	Read	Taint
v4l2_m2m_buffer	DQ→K	Read	Both
v4l2_vp8_entropy	DQ→K	Read	Taint
v4l2_vp8_entropy_coder_state	DQ→K	Read	Taint
v4l2_vp8_loop_filter	DQ→K	Read	Taint
v4l2_vp8_quantization	DQ→K	Read	Taint
v4l2_vp8_segment	DQ→K	Read	Taint
v4l2_vp9_loop_filter	DQ→K	Read	Taint
v4l2_vp9_quantization	DQ→K	Read	Taint
v4l2_vp9_segmentation	DQ→K	Read	Taint
<i>AV1 nested types (syntactic only)</i>			
v4l2_av1_cdef	DQ→K	Read	Syntactic
v4l2_av1_global_motion	DQ→K	Read	Syntactic
v4l2_av1_loop_filter	DQ→K	Read	Syntactic
v4l2_av1_loop_restoration	DQ→K	Read	Syntactic
v4l2_av1_quantization	DQ→K	Read	Syntactic
v4l2_av1_segmentation	DQ→K	Read	Syntactic
v4l2_av1_tile_info	DQ→K	Read	Syntactic
v4l2_ctrl_av1_tile_group_entry	DQ→K	Read	Syntactic
<i>Bidirectional</i>			
vb2_buffer	K↔D	Read	Both
vb2_queue	K↔D	Read/Write	Both
vb2_v4l2_buffer	K↔D	Read/Write	Both

H Static vs. Dynamic Coverage

Table H.1 lists all 367 functions identified by either the CodeQL static analysis or the bpftrace dynamic trace (or both). The *Static* column marks functions included in the CodeQL result set; the *Dynamic* column marks functions observed at runtime during a 307-frame AV1 decode session on the Rock 5B. The *Count* column gives the bpftrace invocation count (zero for static-only entries). Functions are grouped by subsystem.

Subsystem / Function	Static	Dynamic	Count
Subsystem / Function	Static	Dynamic	Count
<i>V4L2 core</i>			
check_fmt	✓	✓	1,281
device_run	✓	✓	307
fh_to_ctx	✓	Skipped	—
get_queue_ctx	✓	✓	1,864
v4l2_apply_frmsize_constraints	✓	✓	20
v4l2_buffer_get_timestamp	✓	Skipped	—
v4l2_buffer_set_timestamp	✓	Skipped	—
v4l2_event_queue_fh	✓	— — —	—
v4l2_event_subscribed	✓	— — —	—
v4l2_fill_pixfmt_mp	✓	✓	13
v4l2_format_block_height	✓	Skipped	—
v4l2_format_block_width	✓	Skipped	—
v4l2_format_info	✓	— — —	—
v4l2_is_colorspace_valid	✓	Skipped	—
v4l2_is_hsv_enc_valid	✓	Skipped	—
v4l2_is_quant_valid	✓	Skipped	—
v4l2_is_xfer_func_valid	✓	Skipped	—
v4l2_is_ycbcr_enc_valid	✓	Skipped	—
v4l2_query_ext_ctrl	✓	✓	17
v4l2_s_ext_ctrls	✓	✓	308
v4l_enable_media_source	✓	✓	6
v4l_fill_fmtdesc	✓	Skipped	—
v4l_pix_format_touch	✓	Skipped	—
v4l_sanitize_colorspace	✓	✓	6
v4l_sanitize_format	✓	✓	6
video_devdata	✓	✓	5,149

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
video_device_node_name	✓	Skipped	—
video_drvdata	✓	Skipped	—
video_get_drvdata	✓	Skipped	—
vidioc_enum_fmt	✓	Skipped	—
vidioc_enum_fmt_vid_cap	✓	✓	12
vidioc_enum_fmt_vid_out	✓	✓	6
vidioc_enum_framesizes	✓	✓	12
vidioc_g_fmt_cap_mplane	✓	✓	6
vidioc_g_fmt_out_mplane	✓	— — —	—
vidioc_querycap	✓	✓	3
vidioc_s_fmt_cap_mplane	✓	— — —	—
vidioc_s_fmt_out_mplane	✓	✓	5
<i>V4L2 M2M</i>			
__v4l2_m2m_try_queue	✓	✓	923
_v4l2_m2m_job_finish	✓	✓	307
v4l2_m2m_adjust_mem_offset	✓	Skipped	—
v4l2_m2m_buf_copy_metadata	✓	✓	307
v4l2_m2m_buf_done	✓	Skipped	—
v4l2_m2m_buf_done_and_job_finish	✓	✓	307
v4l2_m2m_buf_queue	✓	✓	614
v4l2_m2m_buf_remove	✓	✓	616
v4l2_m2m_cancel_job	✓	✓	6
v4l2_m2m_clear_state	✓	Skipped	—
v4l2_m2m_dqbuf	✓	✓	614
v4l2_m2m_dst_buf_is_last	✓	Skipped	—
v4l2_m2m_dst_buf_remove	✓	Skipped	—
v4l2_m2m_expbuf	✓	— — —	—
v4l2_m2m_force_last_buf_done	✓	Skipped	—
v4l2_m2m_get_curr_priv	✓	✓	307
v4l2_m2m_get_vq	✓	✓	18
v4l2_m2m_has_stopped	✓	Skipped	—
v4l2_m2m_ioctl_dqbuf	✓	✓	614
v4l2_m2m_ioctl_expbuf	✓	✓	14
v4l2_m2m_ioctl_qbuf	✓	✓	614
v4l2_m2m_ioctl_querybuf	✓	✓	14
v4l2_m2m_ioctl_reqbufs	✓	✓	4
v4l2_m2m_ioctl_streamoff	✓	✓	2
v4l2_m2m_ioctl_streamon	✓	✓	2
v4l2_m2m_is_last_draining_src_buf	✓	Skipped	—

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
v4l2_m2m_last_buffer_done	✓	— — —	—
v4l2_m2m_mark_stopped	✓	Skipped	—
v4l2_m2m_next_buf	✓	✓	4,049
v4l2_m2m_next_dst_buf	✓	Skipped	—
v4l2_m2m_next_src_buf	✓	Skipped	—
v4l2_m2m_qbuf	✓	✓	614
v4l2_m2m_querybuf	✓	— — —	—
v4l2_m2m_reqbufs	✓	— — —	—
v4l2_m2m_schedule_next_job	✓	Skipped	—
v4l2_m2m_src_buf_remove	✓	Skipped	—
v4l2_m2m_streamoff	✓	✓	2
v4l2_m2m_streamon	✓	— — —	—
v4l2_m2m_try_run	✓	✓	923
v4l2_m2m_try_schedule	✓	— — —	—
v4l2_m2m_update_start_streaming_state	✓	✓	2
v4l2_m2m_update_stop_streaming_state	✓	✓	2
<i>V4L2 controls</i>			
v4l2_ctrl_fill	✓	✓	31
v4l2_ctrl_find	✓	✓	1,228
v4l2_ctrl_get_name	✓	✓	31
v4l2_ctrl_handler_free	✓	✓	311
v4l2_ctrl_handler_free_request	✓	✓	311
v4l2_ctrl_handler_init_class	✓	✓	311
v4l2_ctrl_handler_init_request	✓	✓	311
v4l2_ctrl_lock	✓	Skipped	—
v4l2_ctrl_new	✓	✓	31
v4l2_ctrl_new_std	✓	✓	4
v4l2_ctrl_request_bind	✓	✓	307
v4l2_ctrl_request_clone	✓	Skipped	—
v4l2_ctrl_request_complete	✓	✓	307
v4l2_ctrl_request_setup	✓	✓	307
v4l2_ctrl_request_unbind	✓	✓	307
v4l2_ctrl_type_op_equal	✓	✓	922
v4l2_ctrl_type_op_init	✓	✓	31
v4l2_ctrl_type_op_validate	✓	✓	923
v4l2_ctrl_unlock	✓	Skipped	—
v4l2_ctrls_find_req_obj	✓	Skipped	—
<i>V4L2 control validation</i>			
validate_av1_cdef	✓	Skipped	—

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
validate_av1_film_grain	✓	Skipped	—
validate_av1_frame	✓	Skipped	—
validate_av1_loop_filter	✓	Skipped	—
validate_av1_loop_restoration	✓	Skipped	—
validate_av1_quantization	✓	Skipped	—
validate_av1_segmentation	✓	Skipped	—
validate_av1_sequence	✓	Skipped	—
validate_ctrls	✓	Skipped	—
validate_new	✓	Skipped	—
validate_vp9_compressed_hdr	✓	Skipped	—
validate_vp9_frame	✓	Skipped	—
validate_vp9_lf_params	✓	Skipped	—
validate_vp9_quant_params	✓	Skipped	—
validate_vp9_seg_params	✓	Skipped	—
<i>VB2 core</i>			
__buf_prepare	✓	✓	614
__copy_timestamp	✓	✓	614
__enqueue_in_driver	✓	✓	614
__fill_v4l2_buffer	✓	✓	1,242
__fill_vb2_buffer	✓	✓	614
__init_vb2_v4l2_buffer	✓	✓	628
__vb2_buf_dmabuf_put	✓	— — —	—
__vb2_buf_mem_alloc	✓	Skipped	—
__vb2_buf_mem_finish	✓	Skipped	—
__vb2_buf_mem_free	✓	✓	14
__vb2_buf_mem_prepare	✓	Skipped	—
__vb2_buf_userptr_put	✓	Skipped	—
__vb2_dqbuf	✓	Skipped	—
__vb2_free_mem	✓	Skipped	—
__vb2_get_done_vb	✓	Skipped	—
__vb2_plane_dmabuf_put	✓	Skipped	—
__vb2_queue_alloc	✓	✓	2
__vb2_queue_cancel	✓	✓	12
__vb2_queue_free	✓	✓	10
__vb2_wait_for_done_vb	✓	Skipped	—
__verify_planes_array	✓	✓	628
__verify_planes_array_core	✓	✓	614
vb2_buffer_done	✓	✓	614
vb2_buffer_in_use	✓	✓	1,242

Continued on next page

Table H.1 (*continued*)

Subsystem / Function	Static	Dynamic	Count
vb2_core_allocated_buffers_storage	✓	✓	2
vb2_core_dqbuf	✓	✓	614
vb2_core_expbuf	✓	✓	14
vb2_core_free_buffers_storage	✓	Skipped	—
vb2_core_qbuf	✓	✓	921
vb2_core_querybuf	✓	✓	14
vb2_core_reqbufs	✓	✓	4
vb2_core_streamoff	✓	✓	2
vb2_core_streamon	✓	✓	2
vb2_create_framevec	✓	— — —	—
vb2_destroy_framevec	✓	— — —	—
vb2_dma_contig_plane_dma_addr	✓	Skipped	—
vb2_dqbuf	✓	✓	614
vb2_expbuf	✓	✓	14
vb2_fileio_is_active	✓	Skipped	—
vb2_fill_vb2_v4l2_buffer	✓	Skipped	—
vb2_get_buffer	✓	Skipped	—
vb2_get_drv_priv	✓	Skipped	—
vb2_get_num_buffers	✓	Skipped	—
vb2_get_plane_payload	✓	Skipped	—
vb2_is_busy	✓	Skipped	—
vb2_is_streaming	✓	Skipped	—
vb2_ops_wait_finish	✓	— — —	—
vb2_ops_wait_prepare	✓	— — —	—
vb2_plane_cookie	✓	✓	2,679
vb2_plane_size	✓	Skipped	—
vb2_qbuf	✓	✓	614
vb2_querybuf	✓	✓	14
vb2_queue_add_buffer	✓	Skipped	—
vb2_queue_allows_cache_hints	✓	Skipped	—
vb2_queue_or_prepare_buf	✓	✓	614
vb2_queue_remove_buffer	✓	✓	14
vb2_req_unbind	✓	✓	307
vb2_reqbufs	✓	✓	4
vb2_set_flags_and_caps	✓	✓	4
vb2_set_plane_payload	✓	Skipped	—
vb2_start_streaming	✓	✓	2
vb2_start_streaming_called	✓	Skipped	—
vb2_state_name	✓	Skipped	—
vb2_streamoff	✓	✓	2

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
vb2_streamon	✓	✓	2
vb2_to_hantro_decoded_buf	✓	Skipped	—
vb2_verify_memory_type	✓	✓	4
vb2_warn_zero_bytesused	✓	Skipped	—
<i>VB2 DMA-contig</i>			
vb2_dc_alloc	✓	✓	14
vb2_dc_alloc_coherent	✓	Skipped	—
vb2_dc_alloc_non_coherent	✓	Skipped	—
vb2_dc_attach_dmabuf	✓	— — —	—
vb2_dc_cookie	✓	✓	2,679
vb2_dc_detach_dmabuf	✓	— — —	—
vb2_dc_finish	✓	✓	614
vb2_dc_get_base_sgt	✓	Skipped	—
vb2_dc_get_contiguous_size	✓	Skipped	—
vb2_dc_get_dmabuf	✓	✓	14
vb2_dc_get_userptr	✓	— — —	—
vb2_dc_map_dmabuf	✓	— — —	—
vb2_dc_num_users	✓	✓	1,242
vb2_dc_prepare	✓	✓	614
vb2_dc_put	✓	✓	42
vb2_dc_put_userptr	✓	— — —	—
vb2_dc_unmap_dmabuf	✓	— — —	—
<i>Hantro driver</i>			
hantro_av1_mv_size	✓	Skipped	—
hantro_av1_num_sbs	✓	Skipped	—
hantro_av1_s_ctrl	✓	✓	3
hantro_buf_out_validate	✓	✓	614
hantro_buf_plane_check	✓	Skipped	—
hantro_buf_prepare	✓	✓	614
hantro_buf_queue	✓	✓	614
hantro_buf_request_complete	✓	— — —	—
hantro_check_depth_match	✓	Skipped	—
hantro_end_prepare_run	✓	✓	307
hantro_find_format	✓	✓	50
hantro_get_ctrl	✓	✓	1,228
hantro_get_dec_buf_addr	✓	Skipped	—
hantro_get_default_fmt	✓	✓	14
hantro_get_dst_buf	✓	Skipped	—
hantro_get_format_depth	✓	— — —	—

Continued on next page

Table H.1 (*continued*)

Subsystem / Function	Static	Dynamic	Count
hanthro_get_formats	✓	Skipped	—
hanthro_get_postproc_formats	✓	Skipped	—
hanthro_get_src_buf	✓	Skipped	—
hanthro_h264_mv_size	✓	Skipped	—
hanthro_hevc_chroma_compressed_size	✓	Skipped	—
hanthro_hevc_compressed_size	✓	Skipped	—
hanthro_hevc_luma_compressed_size	✓	Skipped	—
hanthro_hevc_mv_size	✓	Skipped	—
hanthro_irq_done	✓	✓	307
hanthro_job_finish	✓	Skipped	—
hanthro_job_finish_no_pm	✓	Skipped	—
hanthro_needs_postproc	✓	✓	2,701
hanthro_queue_setup	✓	✓	2
hanthro_reg_write	✓	Skipped	—
hanthro_reset_fmt	✓	Skipped	—
hanthro_reset_raw_fmt	✓	✓	11
hanthro_return_bufs	✓	Skipped	—
hanthro_set_fmt_cap	✓	✓	11
hanthro_set_fmt_out	✓	✓	8
hanthro_start_prepare_run	✓	✓	307
hanthro_start_streaming	✓	✓	2
hanthro_stop_streaming	✓	✓	2
hanthro_try_ctrl	✓	✓	3
hanthro_try_fmt	✓	✓	19
hanthro_update_requires_hold_capture_buf	✓	Skipped	—
hanthro_update_requires_request	✓	Skipped	—
hanthro_vp9_mv_size	✓	Skipped	—
hanthro_vp9_num_sbs	✓	Skipped	—
hanthro_vq_is_coded	✓	Skipped	—
hanthro_write_addr	✓	Skipped	—
<i>Rockchip AV1 decoder</i>			
rockchip_av1_default_coeff_probs	✓	✓	44
rockchip_av1_generate_chroma_grain_block	✓	— — —	—
rockchip_av1_generate_luma_grain_block	✓	— — —	—
rockchip_av1_get_cdfs	✓	✓	566
rockchip_av1_get_q_ctx	✓	Skipped	—
rockchip_av1_get_random_number	✓	Skipped	—
rockchip_av1_init_random_generator	✓	Skipped	—
rockchip_av1_set_default_cdfs	✓	✓	1

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
rockchip_av1_store_cdfs	✓	✓	610
rockchip_av1_update_random_register	✓	Skipped	—
rockchip_vpu981_av1_dec_chroma_size	✓	Skipped	—
rockchip_vpu981_av1_dec_clean_refs	✓	Skipped	—
rockchip_vpu981_av1_dec_done	✓	✓	307
rockchip_vpu981_av1_dec_exit	✓	✓	1
rockchip_vpu981_av1_dec_frame_ref	✓	Skipped	—
rockchip_vpu981_av1_dec_frame_unref	✓	Skipped	—
rockchip_vpu981_av1_dec_get_dist	✓	✓	4,480
rockchip_vpu981_av1_dec_get_msb	✓	Skipped	—
rockchip_vpu981_av1_dec_get_shear_params	✓	Skipped	—
rockchip_vpu981_av1_dec_init	✓	✓	1
rockchip_vpu981_av1_dec_init_scaling_function	✓	Skipped	—
rockchip_vpu981_av1_dec_is_lossless	✓	✓	614
rockchip_vpu981_av1_dec_luma_size	✓	Skipped	—
rockchip_vpu981_av1_dec_prepare_run	✓	✓	307
rockchip_vpu981_av1_dec_resolve_divisor_32	✓	Skipped	—
rockchip_vpu981_av1_dec_run	✓	✓	307
rockchip_vpu981_av1_dec_set_cdef	✓	✓	307
rockchip_vpu981_av1_dec_set_fgs	✓	✓	307
rockchip_vpu981_av1_dec_set_frame_sign_bias	✓	Skipped	—
rockchip_vpu981_av1_dec_set_global_model	✓	Skipped	—
rockchip_vpu981_av1_dec_set_input_buffer	✓	Skipped	—
rockchip_vpu981_av1_dec_set_loopfilter	✓	✓	307
rockchip_vpu981_av1_dec_set_lr	✓	Skipped	—
rockchip_vpu981_av1_dec_set_other_frames	✓	✓	295
rockchip_vpu981_av1_dec_set_output_buffer	✓	Skipped	—
rockchip_vpu981_av1_dec_set_parameters	✓	✓	307
rockchip_vpu981_av1_dec_set_picture_dimensions	✓	Skipped	—
rockchip_vpu981_av1_dec_set_prob	✓	Skipped	—
rockchip_vpu981_av1_dec_set_ref	✓	✓	2,065
rockchip_vpu981_av1_dec_set_reference_frames	✓	Skipped	—
rockchip_vpu981_av1_dec_set_segmentation	✓	✓	307
rockchip_vpu981_av1_dec_set_sign_bias	✓	✓	2,065
rockchip_vpu981_av1_dec_set_superres_params	✓	✓	307
rockchip_vpu981_av1_dec_set_tile_info	✓	✓	307
rockchip_vpu981_av1_dec_tiles_free	✓	✓	2
rockchip_vpu981_av1_dec_tiles_reallocate	✓	Skipped	—
rockchip_vpu981_av1_dec_update_prob	✓	✓	307
rockchip_vpu981_av1_tile_log2	✓	Skipped	—

Continued on next page

Table H.1 (*continued*)

Subsystem / Function	Static	Dynamic	Count
rockchip_vpu981_get_frame_index	✓	✓	640
rockchip_vpu981_get_order_hint	✓	✓	1,820
<i>Post-processor</i>			
hantro_postproc_enum_framesizes	✓	✓	2
hantro_postproc_alloc	✓	— — —	—
hantro_postproc_buffer_size	✓	— — —	—
hantro_postproc_disable	✓	✓	307
hantro_postproc_enable	✓	— — —	—
hantro_postproc_free	✓	✓	1
hantro_postproc_get_dec_buf_addr	✓	— — —	—
hantro_postproc_init	✓	— — —	—
rockchip_vpu981_postproc_disable	✓	✓	307
rockchip_vpu981_postproc_enable	✓	— — —	—
<i>Internal helpers</i>			
__buffers_in_use	✓	Skipped	—
__prepare_dmabuf	✓	— — —	—
__prepare_mmap	✓	Skipped	—
__prepare_userptr	✓	— — —	—
__setup_offsets	✓	Skipped	—
__v4l2_event_queue_fh	✓	— — —	—
__verify_dmabuf_ops	✓	Skipped	—
__verify_length	✓	Skipped	—
__verify_mmap_ops	✓	Skipped	—
__verify_userptr_ops	✓	Skipped	—
check_ext_ctrls	✓	— — —	—
check_range	✓	✓	31
clamp	✓	Skipped	—
clamp_roundup	✓	Skipped	—
class_check	✓	Skipped	—
cluster_changed	✓	Skipped	—
cur_to_new	✓	— — —	—
cur_to_req	✓	✓	306
fill_event	✓	Skipped	—
find_private_ref	✓	Skipped	—
find_ref	✓	✓	5,247
find_ref_lock	✓	✓	923
frame_vector_create	✓	— — —	—
frame_vector_destroy	✓	— — —	—
frame_vector_to_pages	✓	— — —	—

Continued on next page

Table H.1 (continued)

Subsystem / Function	Static	Dynamic	Count
frame_vector_to_pfns	✓	— — —	—
get_vaddr_frames	✓	— — —	—
handler_new_ref	✓	✓	1,259
handler_set_err	✓	Skipped	—
init_buffer_cache_hints	✓	Skipped	—
is_cur_manual	✓	Skipped	—
is_new_manual	✓	Skipped	—
new_to_cur	✓	✓	615
new_to_req	✓	✓	922
new_to_user	✓	Skipped	—
node2id	✓	Skipped	—
prepare_ext_ctrls	✓	✓	308
ptr_to_ptr	✓	Skipped	—
ptr_to_user	✓	✓	923
put_vaddr_frames	✓	— — —	—
req_alloc_array	✓	✓	307
req_to_new	✓	✓	922
round_power_of_two	✓	Skipped	—
send_event	✓	✓	615
set_buffer_cache_hints	✓	Skipped	—
set_queue_coherency	✓	Skipped	—
sev_pos	✓	Skipped	—
std_init_compound	✓	Skipped	—
std_validate_compound	✓	Skipped	—
std_validate_elem	✓	Skipped	—
try_or_set_cluster	✓	✓	1,845
try_set_ext_ctrls	✓	✓	308
try_set_ext_ctrls_common	✓	✓	308
try_set_ext_ctrls_request	✓	✓	307
update_from_auto_cluster	✓	— — —	—
user_flags	✓	Skipped	—
user_to_new	✓	✓	923
vdpu_read	✓	Skipped	—
vdpu_read_mask	✓	Skipped	—
vdpu_write	✓	Skipped	—
verify_coherency_flags	✓	Skipped	—

Table H.1: Complete static vs. dynamic function coverage diff. ✓ = present; — = absent; Skipped = not found in available_filter_functions.